STLab02 – Test Case Execution

EduCommerce | Requirement → Test Condition → Test Scenario → Test Case → Test Execution

Mục	Thông tin
Sinh viên / MSSV	Bùi Thanh Bình – 23110134
Lớp / Học phần	23TTH1 – Software Testing
GitHub repository	https://github.com/binhbuithanhk5-a11y/STLab02_EduCommerce
Phạm vi REQ	REQ-02, REQ-04, REQ-07, REQ-09, REQ-10, REQ-15, REQ-16 (7 REQ / 7 module; không chọn REQ-06/08/11)
Môi trường chạy	Python 3.12, pandas, openpyxl, matplotlib (tương thích Google Colab); Gradio cho dashboard
Lần chạy tham chiếu	RUN-20261101-204820 – 38 test case: 34 PASS / 4 FAIL / 0 BLOCKED (pass rate 89.5%)
Defect phát hiện	DEF-001 (Major) – method thanh toán không hợp lệ vẫn trừ tồn kho; DEF-002 (Minor) – Cart.add chấp nhận quantity kiểu bool

Phần	Nội dung	Vị trí trong report
1	Phân tích 7 requirement & thực thi tự động 38 test case bằng Python	Mục 2 – 5
2	Evidence, giải thích code test, root-cause & test report	Mục 6 – 9
3	Problem Solving – 8 tình huống	Mục 10
4	Interview Questions (20 câu) & code-review A/B/C	Mục 11

1–2. Application overview & selected requirements

EduCommerce là ứng dụng in-memory mô phỏng chu trình mua hàng: **Catalog → Cart → Promotions + Shipping → Inventory → Payments → Checkout → Order**. Sản phẩm A (Wireless Mouse 200.000đ, stock 20), B (Mechanical Keyboard 300.000đ, stock 15), C (USB-C Hub 150.000đ, stock 12). Mã nguồn `ecommerce_app.py` được giữ **nguyên** như STLab02 Instruction (không sửa bug trước khi kiểm thử).

Lý do chọn requirement: phủ 7 module khác nhau, ưu tiên các rule có **ranh giới** (REQ-09, REQ-15), **state/side effect** (REQ-07, REQ-10, REQ-15, REQ-16) và **validation input** (REQ-02, REQ-04, REQ-16) – là những nơi rủi ro nghiệp vụ cao nhất ngoài các worked example REQ-06/08/11.

REQ	Module	Acceptance criterion	Vì sao chọn	Số TC
REQ-02	Cart	Số lượng thêm vào giỏ là số nguyên dương; cộng dồn cùng SKU.	Cao – số lượng sai làm sai tiền và tồn kho; nhập trùng SKU tạo nhiều dòng.	8
REQ-04	Authentication	Chỉ người dùng đăng nhập được checkout.	Cao – lộ quy trình đặt hàng cho khách chưa xác thực.	3
REQ-07	Promotion	Voucher không hợp lệ hoặc đã dùng phải bị từ chối; chỉ đánh dấu used sau khi order thành công.	Cao – mất voucher của khách khi thanh toán lỗi hoặc dùng lặp voucher.	6
REQ-09	Shipping	Nội thành miễn ship nếu subtotal ≥ 500.000, nếu không 30.000; ngoại thành 50.000.	Trung bình–cao – sai ranh giới 500k ảnh hưởng mọi đơn.	7
REQ-10	Payment	COD có trạng thái PENDING_COD; ONLINE chỉ PAID khi approved, nếu declined không tạo đơn.	Rất cao – sai trạng thái thanh toán / thu tiền sai.	4
REQ-15	Inventory	Nếu bất kỳ SKU nào không đủ stock, toàn bộ	Rất cao – trừ kho một phần gây lệch tồn.	6

REQ	Module	Acceptance criterion	Vì sao chọn	Số TC
	atomicity	order bị từ chối, không trừ tồn kho phần còn lại.		
REQ-16	Payment safety	Phương thức thanh toán ngoài COD/ONLINE phải bị từ chối.	Cao – request rác gây side effect (giữ kho).	4

3. Requirement analysis, acceptance rules & open questions

Mỗi requirement được viết lại thành **acceptance rule** có thể kiểm chứng (test oracle) trước khi thiết kế test. Expected Result trong CSV được suy ra từ cột "Rule/Oracle" dưới đây, không lấy từ output của ứng dụng.

REQ-02 – Cart

Thuộc tính	Nội dung
Actor	Khách hàng (đã/ chưa đăng nhập)
Rule / Oracle	quantity ∈ ℤ, quantity ≥ 1 → thêm dòng; quantity ≤ 0 / không phải số nguyên → INVALID_QUANTITY, giỏ không đổi. Cùng SKU → 1 dòng, quantity = tổng.
Precondition	Catalog có A/B/C active; giỏ rỗng
Input → Output	sku, quantity → cart.lines, subtotal hoặc ValueError
Risk	Cao – số lượng sai làm sai tiền và tồn kho; nhập trùng SKU tạo nhiều dòng.
Clarification	Có giới hạn số lượng tối đa / theo tồn kho khi add không? Giá trị bool (True) có được coi là số nguyên không? (Tester đề xuất: KHÔNG).

REQ-04 – Authentication

Thuộc tính	Nội dung
Actor	Khách vãng lai / khách đã đăng nhập
Rule / Oracle	authenticated=False → LOGIN_REQUIRED, không tạo order, không trừ kho, không charge, không đánh dấu voucher.
Precondition	Giỏ hợp lệ 600k
Input → Output	customer.authenticated, cart, method → Order hoặc LOGIN_REQUIRED
Risk	Cao – lộ quy trình đặt hàng cho khách chưa xác thực.
Clarification	Session hết hạn giữa chừng xử lý ra sao?

REQ-07 – Promotion

Thuộc tính	Nội dung
Actor	Khách đã đăng nhập
Rule / Oracle	Mã không tồn tại → INVALID_VOUCHER; mã trong used_codes → VOUCHER_USED; used_codes chỉ thêm mã khi place() trả về order.
Precondition	Customer U_NEW, giỏ 600k (≥ ngưỡng REQ-06 để tách biệt lỗi)
Input → Output	voucher, method, approved → discount, used_codes, error
Risk	Cao – mất voucher của khách khi thanh toán lỗi hoặc dùng lặp voucher.
Clarification	Mã có phân biệt hoa/thường không?

REQ-09 – Shipping

Thuộc tính	Nội dung

Thuộc tính	Nội dung
Actor	Khách hàng
Rule / Oracle	inner & subtotal ≥ 500.000 → 0; inner & < 500.000 → 30.000; outer → 50.000 (mọi subtotal); area khác → INVALID_AREA.
Precondition	Giỏ có subtotal theo fixture
Input → Output	subtotal (trước giảm giá), area → shipping, total
Risk	Trung bình–cao – sai ranh giới 500k ảnh hưởng mọi đơn.
Clarification	Subtotal dùng trước hay sau giảm giá? (Instruction 2.4: trước giảm).

REQ-10 – Payment

Thuộc tính	Nội dung
Actor	Khách hàng, cổng thanh toán
Rule / Oracle	COD → status PENDING_COD, không charge; ONLINE+approved → PAID, 1 charge = total; ONLINE+declined → PAYMENT_DECLINED, 0 order, 0 charge.
Precondition	Khách đăng nhập, giỏ hợp lệ
Input → Output	method, approved → order.status, payments.charges
Risk	Rất cao – sai trạng thái thanh toán / thu tiền sai.
Clarification	Charge amount có bằng total sau giảm giá + ship?

REQ-15 – Inventory atomicity

Thuộc tính	Nội dung
Actor	Hệ thống kho
Rule / Oracle	∃ line: qty > stock → OUT_OF_STOCK, stock mọi SKU không đổi, 0 order, 0 charge; qty == stock vẫn hợp lệ.
Precondition	Stock mặc định A=20, B=15, C=12
Input → Output	cart lines nhiều SKU → order / OUT_OF_STOCK, stock sau
Risk	Rất cao – trừ kho một phần gây lệch tồn.
Clarification	Đặt đúng bằng tồn kho có được phép?

REQ-16 – Payment safety

Thuộc tính	Nội dung
Actor	Client/API
Rule / Oracle	method ∉ {COD, ONLINE} → INVALID_PAYMENT_METHOD; không tạo order, không trừ kho, không charge, không đánh dấu voucher.
Precondition	Giỏ 600k, stock đủ
Input → Output	method = WALLET / cod / "" → error + state không đổi
Risk	Cao – request rác gây side effect (giữ kho).
Clarification	"cod" chữ thường có được chuẩn hóa? (Giả định: so khớp chính xác).

Giả định khi chưa có câu trả lời BA: (1) `True/False` không phải số lượng hợp lệ – REQ-02 nói “số nguyên dương” theo nghĩa nghiệp vụ; (2) phương thức thanh toán so khớp chính xác `COD`/`ONLINE` (không tự chuẩn hóa chữ thường); (3) shipping tính trên subtotal trước giảm giá (theo Instruction 2.4); (4) REQ-16 “bị từ chối” bao gồm

không có side effect (không giữ kho, không charge, không tạo order) – suy ra từ REQ-15/REQ-08 về tính nguyên tử của checkout.

4. Test Condition & Test Scenario matrix / traceability

4.1. Test conditions (20 conditions, ≥2/REQ)

REQ	Condition ID	Test condition	Loại
REQ-02	COND-CRT-01	Quantity hợp lệ (biên dưới = 1)	Positive/Boundary
REQ-02	COND-CRT-02	Quantity không hợp lệ: 0, âm, số thực, bool	Negative/Boundary
REQ-02	COND-CRT-03	Cùng SKU thêm nhiều lần được cộng dồn	Positive
REQ-04	COND-AUT-01	Khách đã đăng nhập được checkout	Positive
REQ-04	COND-AUT-02	Khách chưa đăng nhập bị từ chối, không side effect	Negative
REQ-07	COND-VCH-01	Mã không tồn tại bị từ chối	Negative
REQ-07	COND-VCH-02	Mã đã dùng bị từ chối	Negative
REQ-07	COND-VCH-03	Mã chỉ được đánh dấu used khi order thành công	State
REQ-09	COND-SHP-01	Nội thành – quanh ngưỡng 500.000 (499.999 / 500.000 / 500.001)	Boundary
REQ-09	COND-SHP-02	Ngoại thành phí cố định 50.000	Positive
REQ-09	COND-SHP-03	Khu vực không hợp lệ	Negative
REQ-10	COND-PAY-01	COD → PENDING_COD, không charge	Positive
REQ-10	COND-PAY-02	ONLINE approved → PAID, charge = total	Positive
REQ-10	COND-PAY-03	ONLINE declined → không tạo order, không charge	Negative
REQ-15	COND-ATM-01	Một SKU thiếu hàng → reject toàn bộ, stock không đổi	Negative
REQ-15	COND-ATM-02	Số lượng đúng bằng / vượt tồn kho 1 đơn vị	Boundary
REQ-15	COND-ATM-03	Nhiều SKU đủ hàng → trừ đúng từng SKU	Positive
REQ-16	COND-PMT-01	Method lạ (WALLET) bị từ chối	Negative
REQ-16	COND-PMT-02	Method bị từ chối không gây side effect (stock/order/charge)	Negative/State
REQ-16	COND-PMT-03	Giá trị method dị dạng: chữ thường, rỗng	Negative/Boundary

4.2. Test scenarios (18 scenarios, ≥2/REQ)

Scenario ID	REQ	Scenario – WHAT to test	Test cases
TS-CRT-01	REQ-02	Thêm sản phẩm với số lượng hợp lệ nhỏ nhất	TC-CRT-001
TS-CRT-02	REQ-02	Từ chối số lượng không hợp lệ	TC-CRT-002, TC-CRT-003, TC-CRT-004, TC-CRT-005, TC-CRT-008
TS-CRT-03	REQ-02	Thêm cùng SKU nhiều lần được gộp 1 dòng	TC-CRT-006, TC-CRT-007
TS-AUT-01	REQ-04	Khách đã đăng nhập checkout thành công	TC-AUT-001
TS-AUT-02	REQ-04	Khách chưa đăng nhập checkout bị từ chối	TC-AUT-002, TC-AUT-003
TS-VCH-01	REQ-07	Áp mã không hợp lệ	TC-VCH-001
TS-VCH-02	REQ-07	Áp mã đã dùng	TC-VCH-002, TC-VCH-006
TS-VCH-03	REQ-07	Trạng thái voucher sau order thành công / thất bại	TC-VCH-003, TC-VCH-004, TC-VCH-005
TS-SHP-01	REQ-09	Phí ship nội thành tại ngưỡng miễn phí	TC-SHP-001, TC-SHP-002, TC-SHP-003, TC-SHP-004
TS-SHP-02	REQ-09	Phí ship ngoại thành	TC-SHP-005, TC-SHP-006
TS-SHP-03	REQ-09	Khu vực giao hàng không hợp lệ	TC-SHP-007
TS-PAY-01	REQ-10	Đặt hàng COD	TC-PAY-001

Scenario ID	REQ	Scenario – WHAT to test	Test cases
TS-PAY-02	REQ-10	Thanh toán online được duyệt	TC-PAY-002, TC-PAY-004
TS-PAY-03	REQ-10	Thanh toán online bị từ chối	TC-PAY-003
TS-ATM-01	REQ-15	Giỏ nhiều SKU, một SKU thiếu hàng	TC-ATM-001, TC-ATM-002, TC-ATM-005, TC-ATM-006
TS-ATM-02	REQ-15	Đặt hàng ở biên tồn kho	TC-ATM-003, TC-ATM-004
TS-PMT-01	REQ-16	Gửi phương thức thanh toán lạ	TC-PMT-001, TC-PMT-002
TS-PMT-02	REQ-16	Gửi phương thức thanh toán dị dạng	TC-PMT-003, TC-PMT-004

4.3. Traceability matrix REQ → COND → TS → TC → RUN

REQ	COND	TS	TC	RUN 20261101-204820 (P/F/B)
REQ-02	COND-CRT-01	TS-CRT-01	TC-CRT-001	001:P
REQ-02	COND-CRT-02	TS-CRT-02	TC-CRT-002, TC-CRT-003, TC-CRT-004, TC-CRT-005, TC-CRT-008	002:P 003:P 004:P 005:F 008:P
REQ-02	COND-CRT-03	TS-CRT-03	TC-CRT-006, TC-CRT-007	006:P 007:P
REQ-04	COND-AUT-01	TS-AUT-01	TC-AUT-001	001:P
REQ-04	COND-AUT-02	TS-AUT-02	TC-AUT-002, TC-AUT-003	002:P 003:P
REQ-07	COND-VCH-01	TS-VCH-01	TC-VCH-001	001:P
REQ-07	COND-VCH-02	TS-VCH-02	TC-VCH-002, TC-VCH-006	002:P 006:P
REQ-07	COND-VCH-03	TS-VCH-03	TC-VCH-003, TC-VCH-004, TC-VCH-005	003:P 004:P 005:P
REQ-09	COND-SHP-01	TS-SHP-01	TC-SHP-001, TC-SHP-002, TC-SHP-003, TC-SHP-004	001:P 002:P 003:P 004:P
REQ-09	COND-SHP-02	TS-SHP-02	TC-SHP-005, TC-SHP-006	005:P 006:P
REQ-09	COND-SHP-03	TS-SHP-03	TC-SHP-007	007:P
REQ-10	COND-PAY-01	TS-PAY-01	TC-PAY-001	001:P
REQ-10	COND-PAY-02	TS-PAY-02	TC-PAY-002, TC-PAY-004	002:P 004:P
REQ-10	COND-PAY-03	TS-PAY-03	TC-PAY-003	003:P
REQ-15	COND-ATM-01	TS-ATM-01	TC-ATM-001, TC-ATM-002, TC-ATM-006	001:P 002:P 006:P
REQ-15	COND-ATM-02	TS-ATM-02	TC-ATM-003, TC-ATM-004	003:P 004:P
REQ-15	COND-ATM-03	TS-ATM-01	TC-ATM-005	005:P
REQ-16	COND-PMT-01	TS-PMT-01	TC-PMT-001	001:P
REQ-16	COND-PMT-02	TS-PMT-01	TC-PMT-002	002:F
REQ-16	COND-PMT-03	TS-PMT-02	TC-PMT-003, TC-PMT-004	003:F 004:F

Coverage: 7/7 REQ, 20/20 condition, 18/18 scenario có ít nhất 1 test case đã thực thi (không BLOCKED). Ký hiệu RUN: `001:P` = TC-…-001 PASS, `F` = FAIL. Ma trận đầy đủ còn được xuất tự động ở sheet **Traceability** của `reports/test_report.xlsx`.

5. Detailed test cases / test data / expected

38 test case (≥3/REQ), trong đó **20 negative** (expected_type = EXCEPTION) và nhiều boundary (499.999/500.000/500.001; stock 20/21; quantity 0/1). File nguồn: `test_cases.csv` (cột đúng theo yêu cầu 1.5 + cột `test_steps`). Các cột `execution_status`, `actual_result`, `error_message` để trống trong file nguồn; runner ghi kết quả ra file report riêng nên **không ghi đè expected gốc**.

Precondition chung: mỗi case gọi `fixture(fixture_key)` → Checkout/Catalog/Customer(U_NEW, authenticated)/Cart mới; stock mặc định A=20, B=15, C=12. Steps chung theo action: `cart_add`: fixture(empty) → Cart.add() theo thứ tự → so sánh cart_lines/subtotal/error; `place`: fixture → chụp stock → Checkout.place() → chụp orders/charges/stock/used_codes → so sánh; `place_sequence`: nhiều lần place() trên cùng state.

TC ID	REQ / COND / TS	Title	Pri	Fixture	input_json	Type	expected_json
TC-CRT-001	REQ-02 COND-CRT-01 TS-CRT-01	Thêm A số lượng 1 (biên dưới hợp lệ)	P1	empty	{"action": "cart_add", "adds": [["A", 1]]}	RET	{"cart_lines": [["A", 1]], "subtotal": 200000}
TC-CRT-002	REQ-02 COND-CRT-02 TS-CRT-02	Số lượng 0 bị từ chối (biên)	P1	empty	{"action": "cart_add", "adds": [["A", 0]]}	EXC	{"error": "INVALID_QUANTITY", "cart_lines": []}
TC-CRT-003	REQ-02 COND-CRT-02 TS-CRT-02	Số lượng âm bị từ chối	P2	empty	{"action": "cart_add", "adds": [["A", -1]]}	EXC	{"error": "INVALID_QUANTITY", "cart_lines": []}
TC-CRT-004	REQ-02 COND-CRT-02 TS-CRT-02	Số lượng thực 2.5 bị từ chối	P2	empty	{"action": "cart_add", "adds": [["A", 2.5]]}	EXC	{"error": "INVALID_QUANTITY", "cart_lines": []}
TC-CRT-005	REQ-02 COND-CRT-02 TS-CRT-02	Số lượng kiểu bool (true) bị từ chối	P3	empty	{"action": "cart_add", "adds": [["A", true]]}	EXC	{"error": "INVALID_QUANTITY", "cart_lines": []}
TC-CRT-006	REQ-02 COND-CRT-03 TS-CRT-03	Add A=2 rồi A=3 gộp thành A=5	P1	empty	{"action": "cart_add", "adds": [["A", 2], ["A", 3]]}	RET	{"cart_lines": [["A", 5]], "subtotal": 1000000}
TC-CRT-007	REQ-02 COND-CRT-03 TS-CRT-03	A=2, B=1, A=3 → A=5 & B=1, giữ thứ tự	P2	empty	{"action": "cart_add", "adds": [["A", 2], ["B", 1], ["A", 3]]}	RET	{"cart_lines": [["A", 5], ["B", 1]], "subtotal": 1300000}
TC-CRT-008	REQ-02 COND-CRT-02 TS-CRT-02	A=2 rồi A=0: lỗi và dòng A=2 giữ nguyên	P2	empty	{"action": "cart_add", "adds": [["A", 2], ["A", 0]]}	EXC	{"error": "INVALID_QUANTITY", "cart_lines": [["A", 2]], "subtotal": 400000}
TC-AUT-001	REQ-04 COND-AUT-01 TS-AUT-01	Khách đăng nhập đặt COD thành công	P1	600k	{"action": "place", "method": "COD"}	RET	{"status": "PENDING_COD", "orders": 1, "customer_id": "U_NEW"}
TC-AUT-002	REQ-04 COND-AUT-02 TS-AUT-02	Khách chưa đăng nhập đặt COD bị từ chối	P1	600k	{"action": "place", "method": "COD", "authenticated": false}	EXC	{"error": "LOGIN_REQUIRED", "orders": 0, "stock_delta": {}}
TC-AUT-003	REQ-04 COND-AUT-02 TS-AUT-02	Khách chưa đăng nhập + ONLINE + SAVE10: không side effect	P1	600k	{"action": "place", "method": "ONLINE", "approved": true, "voucher": "SAVE10", "authenticated": false}	EXC	{"error": "LOGIN_REQUIRED", "orders": 0, "charges": 0, "stock_delta": {}, "used_codes": []}
TC-VCH-001	REQ-07 COND-VCH-01 TS-VCH-01	Mã SAVE99 không tồn tại bị từ chối	P1	600k	{"action": "place", "voucher": "SAVE99"}	EXC	{"error": "INVALID_VOUCHER", "orders": 0, "stock_delta": {}, "used_codes": []}
TC-VCH-002	REQ-07 COND-VCH-02 TS-VCH-02	SAVE10 đã dùng trước đó bị từ chối	P1	600k	{"action": "place", "voucher": "SAVE10", "pre_used_codes": ["SAVE10"]}	EXC	{"error": "VOUCHER_USED", "orders": 0, "stock_delta": {}}
TC-VCH-003	REQ-07 COND-VCH-03 TS-VCH-03	Order COD thành công → SAVE10 được đánh dấu used	P1	600k	{"action": "place", "voucher": "SAVE10", "method": "COD"}	RET	{"orders": 1, "discount": 60000, "used_codes": ["SAVE10"]}
TC-VCH-004	REQ-07 COND-VCH-03 TS-VCH-03	ONLINE declined → SAVE10 KHÔNG bị đánh dấu used	P0	600k	{"action": "place", "voucher": "SAVE10", "method": "ONLINE", "approved": false}	EXC	{"error": "PAYMENT_DECLINED", "orders": 0, "used_codes": []}
TC-VCH-005	REQ-07 COND-VCH-03 TS-VCH-03	Declined rồi thử lại approved → vẫn dùng được SAVE10	P1	600k	{"action": "place_sequence", "attempts": [{"voucher": "SAVE10", "method": "ONLINE", "approved": false}, {"voucher": "SAVE10", "method": "ONLINE", "approved": true}]}	RET	{"orders": 1, "status": "PAID", "discount": 60000, "used_codes": ["SAVE10"], "attempts": ["#1:ERROR:PAYMENT_DECLINED", "#2:PAID"]}
TC-VCH-006	REQ-07 COND-VCH-02 TS-VCH-02	Declined → approved → dùng lại lần 3 bị VOUCHER_USED	P1	600k	{"action": "place_sequence", "attempts": [{"voucher": "SAVE10", "method": "ONLINE", "approved": false}, {"voucher": "SAVE10", "method": "ONLINE", "approved":	RET	{"orders": 1, "charges": 1, "used_codes": ["SAVE10"], "attempts": ["#1:ERROR:PAYMENT_DECLINED", "#2:PAID", "#3:ERROR:VOUCHER_USED"]}

TC ID	REQ / COND / TS	Title	Pri	Fixture	input_json	Type	expected_json
					true}, {"voucher": "SAVE10", "method": "COD"}]}		
TC-SHP-001	REQ-09 COND-SHP-01 TS-SHP-01	Nội thành 499.999 → ship 30.000	P0	499999	{"action": "place", "area": "inner"}	RET	{"shipping": 30000, "total": 529999}
TC-SHP-002	REQ-09 COND-SHP-01 TS-SHP-01	Nội thành đúng 500.000 → miễn ship	P0	500k	{"action": "place", "area": "inner"}	RET	{"shipping": 0, "total": 500000}
TC-SHP-003	REQ-09 COND-SHP-01 TS-SHP-01	Nội thành 500.001 → miễn ship	P1	500001	{"action": "place", "area": "inner"}	RET	{"shipping": 0, "total": 500001}
TC-SHP-004	REQ-09 COND-SHP-01 TS-SHP-01	Nội thành 600k + SAVE10: ship tính trên subtotal trước giảm	P2	600k	{"action": "place", "area": "inner", "voucher": "SAVE10"}	RET	{"shipping": 0, "discount": 60000, "total": 540000}
TC-SHP-005	REQ-09 COND-SHP-02 TS-SHP-02	Ngoại thành 300k → 50.000	P1	300k	{"action": "place", "area": "outer"}	RET	{"shipping": 50000, "total": 350000}
TC-SHP-006	REQ-09 COND-SHP-02 TS-SHP-02	Ngoại thành 1.500.000 vẫn 50.000 (không miễn)	P1	1500k	{"action": "place", "area": "outer"}	RET	{"shipping": 50000, "total": 1550000}
TC-SHP-007	REQ-09 COND-SHP-03 TS-SHP-03	Khu vực "suburb" bị từ chối, không tạo đơn	P2	600k	{"action": "place", "area": "suburb"}	EXC	{"error": "INVALID_AREA", "orders": 0, "stock_delta": {}}
TC-PAY-001	REQ-10 COND-PAY-01 TS-PAY-01	COD → PENDING_COD, không charge	P0	600k	{"action": "place", "method": "COD"}	RET	{"status": "PENDING_COD", "orders": 1, "charges": 0}
TC-PAY-002	REQ-10 COND-PAY-02 TS-PAY-02	ONLINE approved → PAID, 1 charge = total	P0	600k	{"action": "place", "method": "ONLINE", "approved": true}	RET	{"status": "PAID", "orders": 1, "charges": 1, "charge_amount": 600000}
TC-PAY-003	REQ-10 COND-PAY-03 TS-PAY-03	ONLINE declined → không order, không charge	P0	600k	{"action": "place", "method": "ONLINE", "approved": false}	EXC	{"error": "PAYMENT_DECLINED", "orders": 0, "charges": 0}
TC-PAY-004	REQ-10 COND-PAY-02 TS-PAY-02	ONLINE approved 1.5M + SAVE10 → charge 1.400.000	P1	1500k	{"action": "place", "method": "ONLINE", "approved": true, "voucher": "SAVE10"}	RET	{"status": "PAID", "charges": 1, "charge_amount": 1400000, "total": 1400000}
TC-ATM-001	REQ-15 COND-ATM-01 TS-ATM-01	A=2 đủ, B=100 thiếu (COD) → reject, A không bị trừ	P0	A2_B100	{"action": "place", "method": "COD"}	EXC	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}
TC-ATM-002	REQ-15 COND-ATM-01 TS-ATM-01	A=2, B=100 (ONLINE approved) → không charge, stock không đổi	P0	A2_B100	{"action": "place", "method": "ONLINE", "approved": true}	EXC	{"error": "OUT_OF_STOCK", "orders": 0, "charges": 0, "stock_delta": {}}
TC-ATM-003	REQ-15 COND-ATM-02 TS-ATM-02	A=20 đúng bằng tồn kho → thành công, A về 0	P1	A20	{"action": "place", "method": "COD"}	RET	{"orders": 1, "stock_delta": {"A": -20}}
TC-ATM-004	REQ-15 COND-ATM-02 TS-ATM-02	A=21 vượt tồn kho 1 → OUT_OF_STOCK	P1	A21	{"action": "place", "method": "COD"}	EXC	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}
TC-ATM-005	REQ-15 COND-ATM-03 TS-ATM-01	A=2, B=1 đủ hàng → trừ đúng từng SKU	P2	A2_B1	{"action": "place", "method": "COD"}	RET	{"orders": 1, "stock_delta": {"A": -2, "B": -1}}
TC-ATM-006	REQ-15 COND-ATM-01 TS-ATM-01	Đảo thứ tự: B=100 (thiếu) trước, A=2 sau → stock không đổi	P1	B100_A2	{"action": "place", "method": "COD"}	EXC	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}
TC-PMT-001	REQ-16 COND-PMT-01 TS-PMT-01	method=WALLET → INVALID_PAYMENT_METHOD, không tạo đơn	P0	600k	{"action": "place", "method": "WALLET"}	EXC	{"error": "INVALID_PAYMENT_METHOD", "orders": 0}
TC-PMT-002	REQ-16 COND-PMT-02 TS-PMT-01	method=WALLET không trừ kho, không charge, không dùng voucher	P0	600k	{"action": "place", "method": "WALLET", "voucher": "SAVE10"}	EXC	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "charges": 0, "stock_delta": {},

TC ID	REQ / COND / TS	Title	Pri	Fixture	input_json	Type	expected_json
							`"used_codes": []}`
TC-PMT-003	REQ-16 COND-PMT-03 TS-PMT-02	method="cod" (chữ thường) bị từ chối, stock không đổi	P2	600k	`{"action": "place", "method": "cod"}`	EXC	`{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "stock_delta": {}}`
TC-PMT-004	REQ-16 COND-PMT-03 TS-PMT-02	method rỗng "" bị từ chối, stock không đổi	P2	600k	`{"action": "place", "method": ""}`	EXC	`{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "stock_delta": {}}`

5.1. Ví dụ chi tiết 3 test case

Trường	TC-SHP-002
Trace	REQ-09 / COND-SHP-01 / TS-SHP-01
Precondition	Fixture mới: Checkout/Catalog/Customer/Cart riêng; customer U_NEW đăng nhập
Test data	{"action": "place", "area": "inner"}
Steps	1. fixture("500k") → giỏ A×1 + B×1 = 500.000. 2. Checkout.place(area="inner", method="COD"). 3. Đọc order.shipping, order.total.
Expected	shipping = 0; total = 500.000 (đúng ngưỡng "≥ 500.000" → miễn ship).
Actual / Status	Ghi tự động khi chạy – PASS (RUN-20261101-204820)

Trường	TC-PMT-002
Trace	REQ-16 / COND-PMT-02 / TS-PMT-01
Precondition	Fixture mới: Checkout/Catalog/Customer/Cart riêng; customer U_NEW đăng nhập
Test data	{"action": "place", "method": "WALLET", "voucher": "SAVE10"}
Steps	1. fixture("600k") → A×3; chụp stock {A:20,B:15,C:12}. 2. place(method="WALLET", voucher="SAVE10"). 3. Chụp stock, len(orders), len(charges), used_codes.
Expected	ValueError INVALID_PAYMENT_METHOD; orders = 0; charges = 0; stock_delta = {} (stock trước = sau); used_codes = [].
Actual / Status	Ghi tự động khi chạy – FAIL (RUN-20261101-204820)

Trường	TC-VCH-005
Trace	REQ-07 / COND-VCH-03 / TS-VCH-03
Precondition	Fixture mới: Checkout/Catalog/Customer/Cart riêng; customer U_NEW đăng nhập
Test data	{"action": "place_sequence", "attempts": [{"voucher": "SAVE10", "method": "ONLINE", "approved": false}, {"voucher": "SAVE10", "method": "ONLINE", "approved": true}]}
Steps	1. fixture("600k"). 2. place(SAVE10, ONLINE, approved=False) → kỳ vọng PAYMENT_DECLINED. 3. place(SAVE10, ONLINE, approved=True) trên cùng customer/cart.
Expected	Lần 1 lỗi PAYMENT_DECLINED và used_codes vẫn rỗng; lần 2 PAID, discount 60.000, used_codes = [SAVE10], orders = 1.
Actual / Status	Ghi tự động khi chạy – PASS (RUN-20261101-204820)

6. Python test architecture

Luồng thực thi (STEP 5 của yêu cầu): **CSV/Excel → validate → fixture() → dispatcher gọi function → quan sát return + state → so expected/actual → PASS/FAIL/BLOCKED → evidence → report**. Toàn bộ nằm trong `test_runner.py` (~250 dòng).

```
test_cases.csv ──read_cases()──► list[row] ──► execute_case(row) ×N ──► DataFrame
   (schema, dup-ID check)          │ 1. parse input_json/expected_json   (lỗi → BLOCKED)
                                   │ 2. fixture(fixture_key)  → Checkout/Customer/Cart MỚI
                                   │ 3. act_cart_add / act_place / act_place_sequence
                                   │ 4. snapshot(): orders, charges, stock_delta, used_codes,
cart_lines…
                                   │ 5. so khớp từng khóa expected_json  → PASS / FAIL
                                   ▼
run_suite(): completeness check → test_execution.csv, test_report.xlsx (4 sheet), test_dashboard.html,
chart PNG
```

6.1. fixture() – môi trường độc lập (E03)

test_runner.py — FIXTURES + fixture()

```python
FIXTURES = {
    'empty':   {'lines': []},                                          # empty cart
    '300k':    {'lines': [('B', 1)]},                                  # 300.000
    '499999':  {'lines': [('A', 1), ('B', 1)], 'price': {'B': 299999}},  # just below 500k
    '500k':    {'lines': [('A', 1), ('B', 1)]},                        # exactly 500.000
    '500001':  {'lines': [('A', 1), ('B', 1)], 'price': {'B': 300001}},  # just above 500k
    '600k':    {'lines': [('A', 3)]},                                  # 600.000
    '1500k':   {'lines': [('B', 5)]},                                  # 1.500.000
    'A2_B1':   {'lines': [('A', 2), ('B', 1)]},                        # 2 SKUs, enough stock
    'A2_B100': {'lines': [('A', 2), ('B', 100)]},                      # B stock 15 < 100
    'B100_A2': {'lines': [('B', 100), ('A', 2)]},                      # same, short SKU first
    'A20':     {'lines': [('A', 20)]},                                 # == stock of A
    'A21':     {'lines': [('A', 21)]},                                 # stock of A + 1
}


def fixture(key, authenticated=True, used_codes=()):
    if key not in FIXTURES:
        raise BlockedError(f'UNKNOWN_FIXTURE:{key}')
    spec = FIXTURES[key]
    service = Checkout()
    customer = Customer('U_NEW', authenticated=authenticated, used_codes=set(used_codes))
    cart = Cart(service.catalog)
    for sku, price in spec.get('price', {}).items():
        service.catalog.products[sku].price = D(price)
    for sku, qty in spec['lines']:
        # Cart lines bypass Cart.add() so a fixture never depends on the code under test.
        cart.lines.append(CartLine(sku, qty))
    return service, customer, cart
```

Hình 1. Bảng FIXTURES và hàm fixture() trong test_runner.py

- Mỗi lần gọi `fixture(key)` tạo **Checkout() mới** (kéo theo Catalog, Inventory, Promotions, Shipping, Payments, `orders`, `by_key` mới), **Customer("U_NEW") mới** (tham số `authenticated`, `used_codes` lấy từ input_json) và **Cart() mới** gắn với catalog của chính engine đó.
- Ý nghĩa từng key: `empty` (giỏ rỗng cho test Cart.add), `300k`=B×1, `499999`=A×1+B×1 với giá B 299.999, `500k`=A×1+B×1, `500001`=giá B 300.001, `600k`=A×3, `1500k`=B×5, `A2_B1`, `A2_B100`/`B100_A2` (B thiếu hàng, hai thứ tự), `A20` (= tồn kho), `A21` (tồn kho + 1).
- Dòng giỏ được nạp bằng `CartLine` trực tiếp thay vì `Cart.add()` để precondition **không phụ thuộc** vào chính code đang bị kiểm thử (REQ-02 có bug thì các REQ khác không bị kéo theo).
- Key lạ → `BlockedError(UNKNOWN_FIXTURE)` → case BLOCKED, tránh chạy nhầm dữ liệu. Việc đổi giá B ở fixture `499999` chỉ ảnh hưởng catalog của case đó (chứng minh ở Hình 8).

6.2. Đọc CSV/Excel & kiểm tra input (E04)

test_runner.py — read_cases(): CSV/XLSX + schema validation

```python
def read_cases(path):
    path = Path(path)
    if path.suffix.lower() in ('.xlsx', '.xls'):
        df = pd.read_excel(path, dtype=str)
    else:
        df = pd.read_csv(path, dtype=str, encoding='utf-8-sig', keep_default_na=False)
    df = df.fillna('')
    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(f'MISSING_COLUMNS:{missing}')
    dup = df.test_case_id[df.test_case_id.duplicated()].tolist()
    if dup:
        raise ValueError(f'DUPLICATE_TEST_IDS:{dup}')
    return df.to_dict('records')
```

Hình 2. read_cases(): pandas.read_csv / read_excel + schema validation

- `pd.read_csv(dtype=str, keep_default_na=False)` đọc mọi cột dạng chuỗi (không để pandas tự đổi `""` thành NaN hay `0` thành số); file `.xlsx` dùng `pd.read_excel` – cùng một runner chạy được cả hai định dạng.
- Kiểm tra đủ 15 cột bắt buộc (`MISSING_COLUMNS`) và **trùng test_case_id** (`DUPLICATE_TEST_IDS`) – vi phạm thì dừng cả suite vì kết quả sẽ không truy vết được.
- `test_case_id` là khóa của mỗi dòng kết quả; `requirement_id/condition_id/scenario_id` được giữ nguyên sang report để dựng traceability và tổng hợp theo REQ.

test_runner.py — execute_case(): parse & validate input (BLOCKED)

```python
        # ---- 1. setup (problems here => BLOCKED, never FAIL)
        try:
            inp = json.loads(row['input_json'] or '{}')
            expected = json.loads(row['expected_json'])
        except json.JSONDecodeError as exc:
            raise BlockedError(f'INVALID_JSON:{exc.msg}')
        etype = row['expected_type'].strip().upper()
        if etype not in EXPECTED_TYPES:
            raise BlockedError(f'UNKNOWN_EXPECTED_TYPE:{etype}')
        action = inp.get('action')
        if action not in ('cart_add', 'place', 'place_sequence'):
            raise BlockedError(f'UNKNOWN_TEST_ACTION:{action}')
        app, customer, cart = fixture(row['fixture_key'], inp.get('authenticated', True),
                                      inp.get('pre_used_codes', []))
        before = stock_of(app)
```

Hình 3. execute_case() – bước 1: parse & validate test data trước khi chạy

- `input_json`/`expected_json` được `json.loads`; JSON hỏng → BLOCKED `INVALID_JSON`. `expected_type` phải là RETURN/EXCEPTION; `action` phải thuộc dispatcher – nếu không → BLOCKED (không tự đoán logic).
- Chỉ khi setup thành công mới chụp `before = stock_of(app)` làm mốc so sánh state.

6.3. Dispatcher – gọi function nghiệp vụ & so sánh (E05)

test_runner.py — actions calling business functions

```python
def act_cart_add(app, customer, cart, inp):
    """REQ-02: call Cart.add() for each [sku, qty] in order. Stops at first exception."""
    for sku, qty in inp['adds']:
        cart.add(sku, qty)
    return None


def act_place(app, customer, cart, inp, key):
    """Call Checkout.place() once with the arguments from input_json."""
    return app.place(customer, cart, area=inp.get('area', 'inner'), voucher=inp.get('voucher'),
                     method=inp.get('method', 'COD'), approved=inp.get('approved', True), key=key)


def act_place_sequence(app, customer, cart, inp, key):
    """Several place() attempts on the SAME fixture (e.g. declined then approved).
    Only the last attempt's exception is reported; earlier expected failures are recorded."""
    order, notes = None, []
    for i, attempt in enumerate(inp['attempts'], 1):
        try:
            order = act_place(app, customer, cart, attempt, f'{key}-{i}')
            notes.append(f'#{i}:{order.status}')
        except ValueError as exc:
            notes.append(f'#{i}:ERROR:{exc}')
    return order, notes
```

Hình 4. Các action map tới function của ứng dụng

- `cart_add` → `Cart.add(sku, qty)` lần lượt cho từng cặp (REQ-02); exception đầu tiên dừng chuỗi để kiểm state sau lỗi (TC-CRT-008).
- `place` → `Checkout.place(customer, cart, area, voucher, method, approved, key=test_case_id)` – dùng cho REQ-04/07/09/10/15/16; key = TC ID để không đụng idempotency.
- `place_sequence` → nhiều lần `place()` trên **cùng** state (declined → approved → reuse) để kiểm chuyển trạng thái voucher (REQ-07); kết quả từng lần lưu ở `attempts`.

test_runner.py — dispatch + observe + compare expected/actual

```
138         # ---- 2. execute the function under test
139         order, error, notes = None, '', []
140         try:
141             if action == 'cart_add':
142                 act_cart_add(app, customer, cart, inp)
143             elif action == 'place':
144                 order = act_place(app, customer, cart, inp, case_id)
145             else:
146                 order, notes = act_place_sequence(app, customer, cart, inp, case_id)
147         except ValueError as exc:            # business rule rejection raised by the app
148             error = str(exc)
149         except Exception as exc:             # crash inside the app = unexpected => FAIL
150             error = f'UNEXPECTED:{type(exc).__name__}:{exc}'
151
152         # ---- 3. observe return value + state, then compare
153         obs = snapshot(app, customer, cart, before, order)
154         obs['error'] = error
155         if notes:
156             obs['attempts'] = notes
157         if etype == 'RETURN' and 'error' not in expected:
158             expected = {**expected, 'error': ''}      # a RETURN case must not raise
159         mismatches = {k: {'expected': v, 'actual': obs.get(k, '<missing>')}
160                       for k, v in expected.items() if obs.get(k, '<missing>') != v}
161         result.update({
162             'execution_status': 'PASS' if not mismatches else 'FAIL',
163             'actual_result': json.dumps({k: obs.get(k, '<missing>') for k in expected}, ensure_ascii=False),
164             'error_message': error if not mismatches else json.dumps(mismatches, ensure_ascii=False),
165             'observed_json': json.dumps(obs, ensure_ascii=False, default=str),
166             'stock_before': json.dumps(before), 'stock_after': json.dumps(stock_of(app)),
167         })
```

Hình 5. execute_case() – bước 2–3: thực thi, quan sát return + state, so expected/actual

- `snapshot()` thu **cả kết quả trả về** (status, discount, shipping, total, customer_id) **và state sau thao tác**: `orders`, `charges`, `charge_amount`, `stock_delta` (chỉ SKU thay đổi), `used_codes`, `cart_lines`, `subtotal`.
- So sánh theo từng khóa có trong `expected_json` (oracle chỉ khai báo những gì requirement quy định). Case RETURN tự thêm `error: ""` → nếu ứng dụng ném lỗi bất ngờ thì FAIL.
- Khi FAIL, `error_message` chứa JSON mismatch `{khóa: {expected, actual}}` – đây là evidence chính để phân tích (Hình 15).

6.4. Exception handling & trạng thái PASS/FAIL/BLOCKED (E06)

test_runner.py — try/except + PASS/FAIL/BLOCKED

```python
def execute_case(row):
    """Run one CSV row. Returns the row enriched with execution_status/actual_result/..."""
    started = time.perf_counter()
    case_id = row.get('test_case_id', '')
    result = {'execution_status': 'BLOCKED', 'actual_result': '', 'error_message': '',
              'observed_json': '', 'stock_before': '', 'stock_after': ''}
    try:
        # ---- 1. setup (problems here => BLOCKED, never FAIL)
        try:
            inp = json.loads(row['input_json'] or '{}')
            expected = json.loads(row['expected_json'])
        except json.JSONDecodeError as exc:
            raise BlockedError(f'INVALID_JSON:{exc.msg}')
        etype = row['expected_type'].strip().upper()
        if etype not in EXPECTED_TYPES:
            raise BlockedError(f'UNKNOWN_EXPECTED_TYPE:{etype}')
        action = inp.get('action')
        if action not in ('cart_add', 'place', 'place_sequence'):
            raise BlockedError(f'UNKNOWN_TEST_ACTION:{action}')
        app, customer, cart = fixture(row['fixture_key'], inp.get('authenticated', True),
                                      inp.get('pre_used_codes', []))
        before = stock_of(app)

        # ---- 2. execute the function under test
        order, error, notes = None, '', []
        try:
            if action == 'cart_add':
                act_cart_add(app, customer, cart, inp)
            elif action == 'place':
                order = act_place(app, customer, cart, inp, case_id)
            else:
                order, notes = act_place_sequence(app, customer, cart, inp, case_id)
        except ValueError as exc:            # business rule rejection raised by the app
            error = str(exc)
        except Exception as exc:             # crash inside the app = unexpected => FAIL
            error = f'UNEXPECTED:{type(exc).__name__}:{exc}'

        # ---- 3. observe return value + state, then compare
        obs = snapshot(app, customer, cart, before, order)
        obs['error'] = error
        if notes:
            obs['attempts'] = notes
        if etype == 'RETURN' and 'error' not in expected:
            expected = {**expected, 'error': ''}      # a RETURN case must not raise
        mismatches = {k: {'expected': v, 'actual': obs.get(k, '<missing>')}
                      for k, v in expected.items() if obs.get(k, '<missing>') != v}
        result.update({
            'execution_status': 'PASS' if not mismatches else 'FAIL',
            'actual_result': json.dumps({k: obs.get(k, '<missing>') for k in expected}, ensure_ascii=False),
            'error_message': error if not mismatches else json.dumps(mismatches, ensure_ascii=False),
            'observed_json': json.dumps(obs, ensure_ascii=False, default=str),
            'stock_before': json.dumps(before), 'stock_after': json.dumps(stock_of(app)),
        })
    except BlockedError as exc:
        result['error_message'] = str(exc)
    except Exception as exc:                 # bug in the test harness itself
        result['error_message'] = f'HARNESS_ERROR:{type(exc).__name__}:{exc}'
        result['traceback'] = traceback.format_exc(limit=3)
    result['duration_ms'] = round((time.perf_counter() - started) * 1000, 3)
    result['executed_at'] = datetime.datetime.now(datetime.timezone.utc).isoformat(timespec='seconds')
    return {**row, **result}
```

Hình 6. execute_case() đầy đủ: try/except phân tầng

- **ValueError từ ứng dụng** = phản hồi nghiệp vụ (vd. `OUT_OF_STOCK`) → ghi vào `obs.error` rồi so với expected. Đây là cách phân biệt "exception mong đợi → PASS" và "exception sai/không có → FAIL".
- **Exception khác từ ứng dụng** (TypeError, KeyError…) → `UNEXPECTED:<type>` → luôn FAIL (crash).

- **BlockedError** (lỗi dữ liệu/setup) → BLOCKED; **lỗi trong chính runner** → BLOCKED kèm `HARNESS_ERROR` + traceback – không đổ lỗi cho sản phẩm.
- Mỗi case ghi `duration_ms` (perf_counter) và `executed_at` (UTC ISO-8601) làm execution time/timestamp.

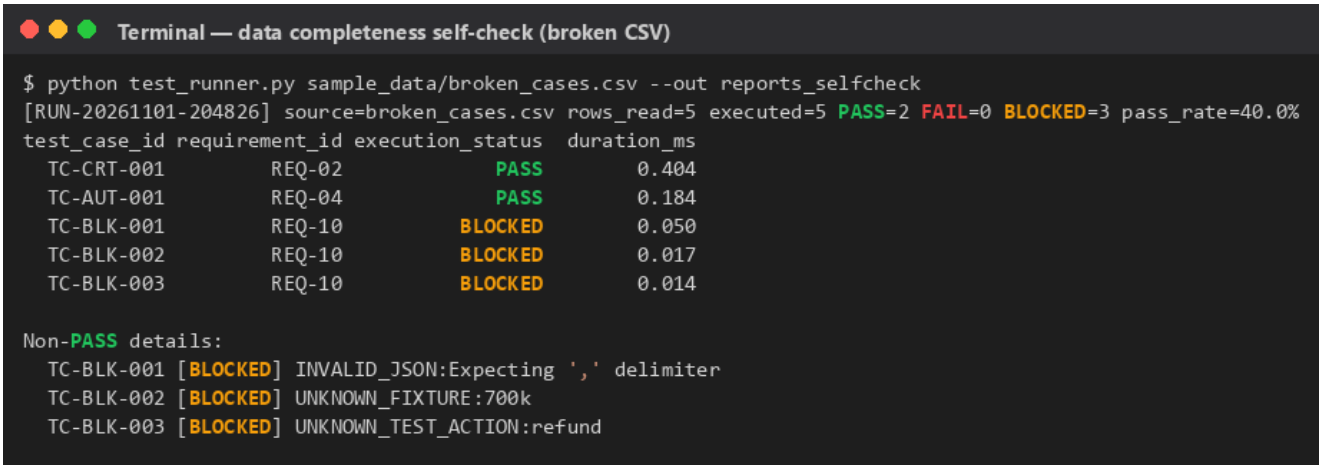
6.5. Reporter

```
counts  = df.execution_status.value_counts().reindex(['PASS','FAIL','BLOCKED'], fill_value=0)
assert len(df) == len(cases)                    # data completeness: rows read == rows reported
pass_rate = 100 * PASS / len(df)                # mẫu số = TẤT CẢ dòng đọc được (gồm BLOCKED)
by_req    = groupby(requirement_id, status) + TOTAL + PASS_RATE_%
coverage  = REQ có ≥1 case không BLOCKED / tổng REQ
→ test_execution.csv | test_report.xlsx (Summary, ByRequirement, Traceability, Execution)
→ test_summary_chart.png (matplotlib) | test_dashboard.html (metric + chart + bảng) |
dashboard_gradio.py
```

Pass rate dùng mẫu số là **tổng số dòng đọc được**, nên dòng BLOCKED kéo pass rate xuống thay vì "biến mất" (xem 3.7). Dashboard Gradio (`dashboard_gradio.py`) đặt chart cạnh bảng, lọc theo REQ/status, có nút tải Excel.

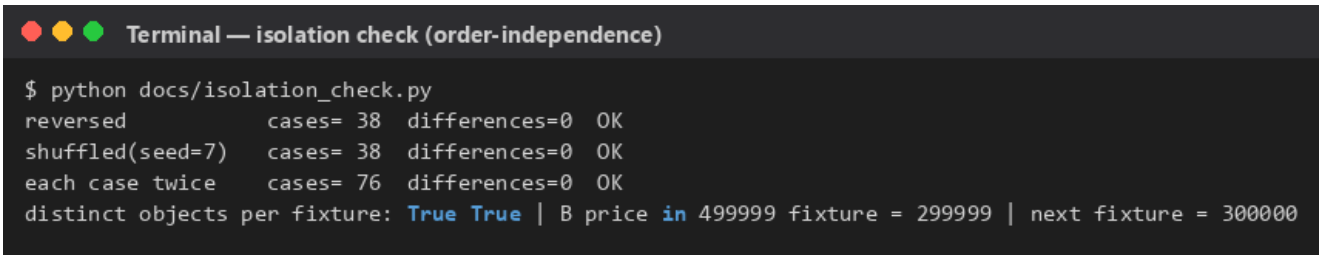
7. Execution evidence

Evidence	Nội dung	Vị trí
E01	Cấu trúc source	Hình 9
E02	Bảng REQ / condition / scenario / TC / traceability	Mục 2–5
E03	fixture()	Hình 1
E04	CSV/Excel reader + parse input	Hình 2–3
E05	Dispatcher + compare	Hình 4–5
E06	try/except + PASS/FAIL/BLOCKED	Hình 6
E07	Output chạy đủ bộ test + timestamp	Hình 10, 7, 8
E08	Bảng kết quả + chart theo REQ/status	Hình 12–14
E09	FAIL: TC, expected/actual, code nghi vấn, tái hiện	Hình 15–19
E10	README / cách chạy từ môi trường sạch	Hình 11
E11–E12	Isolation check, retest sau khi vá (bổ sung)	Hình 8, 20



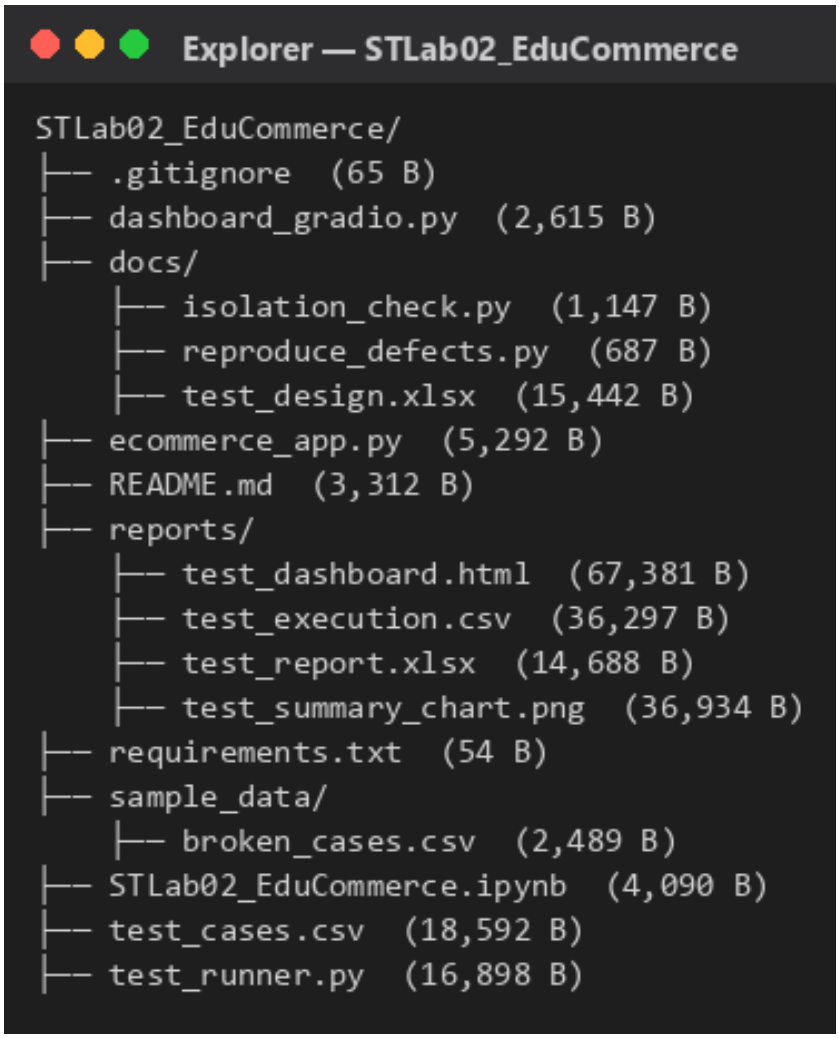
Hình 7. Data completeness: CSV có 3 dòng lỗi → 3 BLOCKED, không bị bỏ qua

- File `sample_data/broken_cases.csv` (5 dòng) cố ý chứa JSON hỏng, fixture `700k` không tồn tại và action `refund`. Runner vẫn báo đủ **rows_read=5 = executed=5**, pass rate 40% chứ không phải 100%.



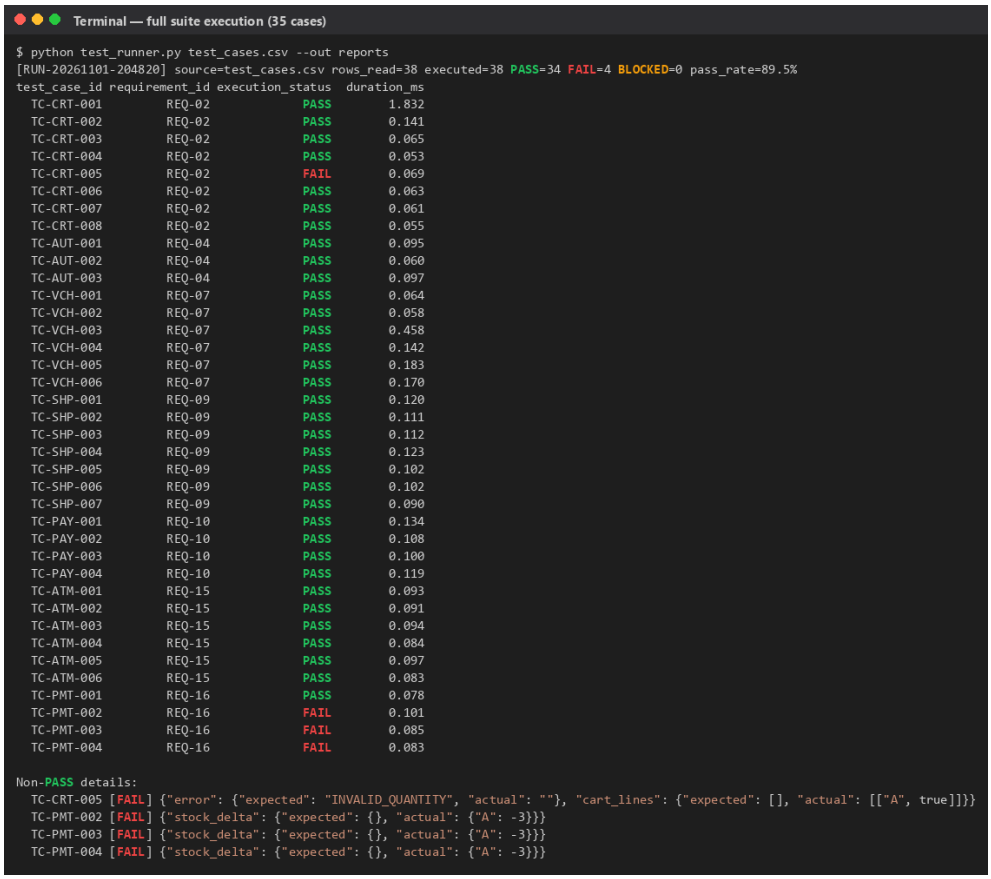
Hình 8. Isolation check: chạy theo thứ tự gốc, đảo ngược, xáo trộn và lặp đôi – 0 khác biệt

- `docs/isolation_check.py` chứng minh kết quả không phụ thuộc thứ tự: mỗi fixture là object riêng (`a.catalog is not b.catalog`), giá B = 299.999 ở fixture `499999` không rò sang fixture kế tiếp.



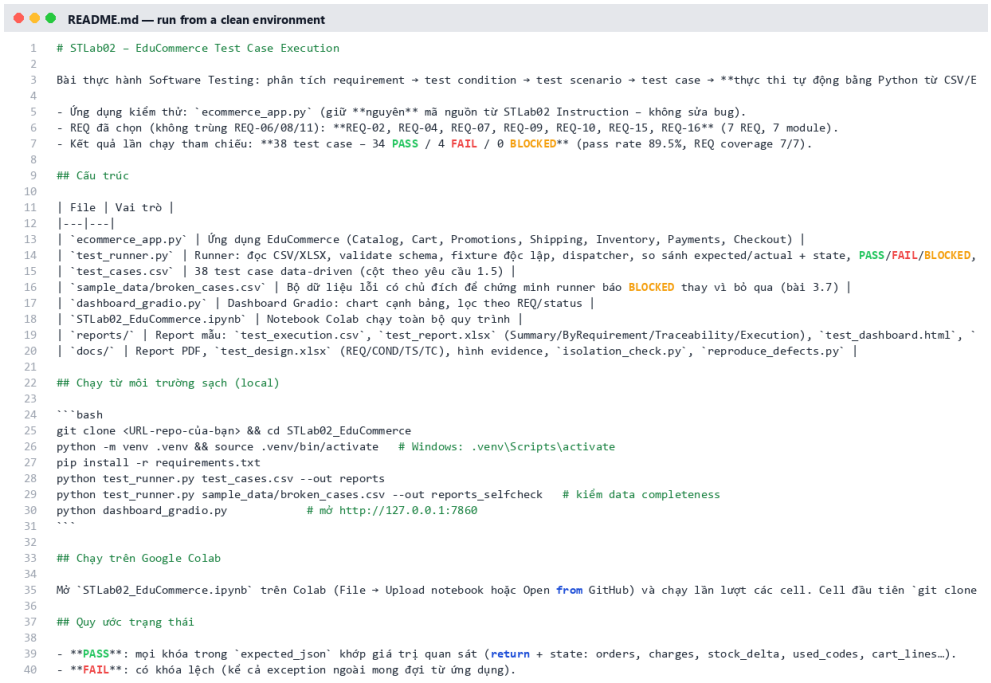
Hình 9. E01 – Cấu trúc repository

- `ecommerce_app.py` (SUT, giữ nguyên), `test_runner.py` (runner), `test_cases.csv` (dữ liệu), `STLab02_EduCommerce.ipynb` (Colab), `dashboard_gradio.py`, `README.md`, `requirements.txt`, `reports/` (report mẫu), `docs/` (design, evidence, script tái hiện).



Hình 10. E07 – Output chạy toàn bộ 38 test case (RUN-20261101-204820)

- Dòng tổng: rows_read = executed = 38; **PASS 34 / FAIL 4 / BLOCKED 0**; pass rate 89.5%. Mỗi dòng có thời gian thực thi (ms); timestamp UTC nằm ở cột `executed_at` trong CSV.
- Notebook `STLab02_EduCommerce.ipynb` chạy cùng lệnh `run_suite()` trên Colab (cell in thời điểm chạy + phiên bản Python).



Hình 11. E10 – README: cách chạy từ môi trường sạch (local & Colab)

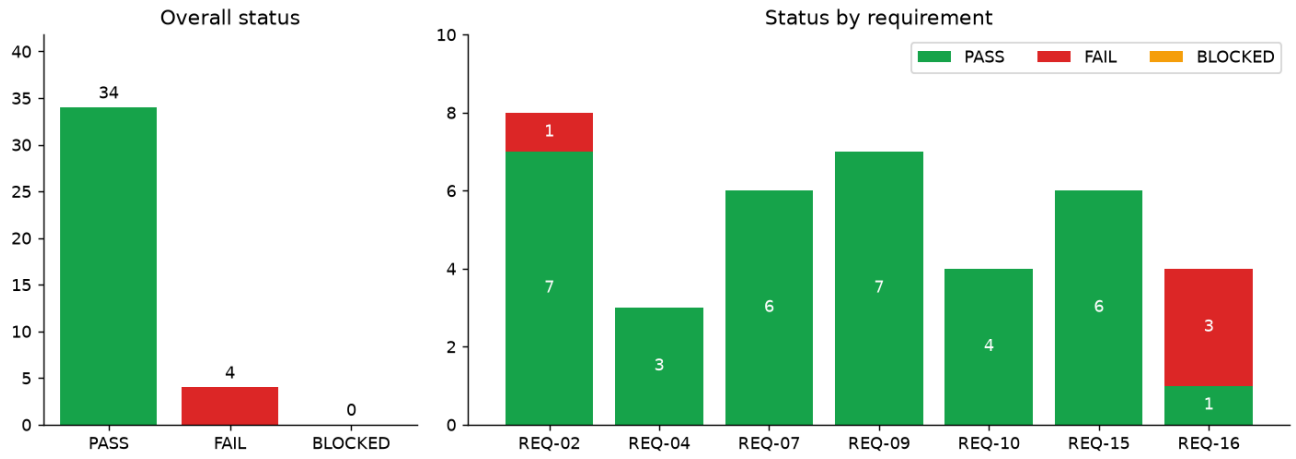
- README mô tả cấu trúc, lệnh cài đặt, chạy suite, self-check, dashboard; notebook clone repo nên chạy được từ runtime Colab mới.

Ghi chú: ảnh output trên được chụp từ lần chạy local (Python 3.12). Khi nộp, sinh viên chạy notebook trên Colab và bổ sung ảnh chụp màn hình Colab (cell output + timestamp) vào vị trí này.

8. Test report: bảng, chart, tổng hợp theo requirement, coverage

Chỉ số	Giá trị
Run ID	RUN-20261101-204820
Rows read / executed	38 / 38
PASS / FAIL / BLOCKED	34 / 4 / 0
Pass rate	89.5%
Requirement coverage	7/7 (100%)
Tổng thời gian thực thi	5.78 ms

requirement_id	PASS	FAIL	BLOCKED	TOTAL	PASS_RATE_%
REQ-02	7	1	0	8	87.5
REQ-04	3	0	0	3	100.0
REQ-07	6	0	0	6	100.0
REQ-09	7	0	0	7	100.0
REQ-10	4	0	0	4	100.0
REQ-15	6	0	0	6	100.0
REQ-16	1	3	0	4	25.0



Hình 12. E08 – Chart tổng hợp theo trạng thái và theo requirement (reports/test_summary_chart.png)

- Hai REQ có FAIL: REQ-16 (3/4 FAIL – side effect tồn kho) và REQ-02 (1/8 FAIL – quantity bool). Các REQ còn lại 100% PASS, bao gồm các biên shipping và tính nguyên tử khi thiếu hàng.

reports/test_execution.csv — RUN-20261101-204820

TC	REQ	COND	TS	Expected (expected_json)	Actual (actual_result)	Status	ms
TC-CRT-001	REQ-02	COND-CRT-01	TS-CRT-01	{"cart_lines": [["A", 1]], "subtotal": 200000}	{"cart_lines": [["A", 1]], "subtotal": 200000, "error": ""}	PASS	1.832
TC-CRT-002	REQ-02	COND-CRT-02	TS-CRT-02	{"error": "INVALID_QUANTITY", "cart_lines": []}	{"error": "INVALID_QUANTITY", "cart_lines": []}	PASS	0.141
TC-CRT-003	REQ-02	COND-CRT-02	TS-CRT-02	{"error": "INVALID_QUANTITY", "cart_lines": []}	{"error": "INVALID_QUANTITY", "cart_lines": []}	PASS	0.065
TC-CRT-004	REQ-02	COND-CRT-02	TS-CRT-02	{"error": "INVALID_QUANTITY", "cart_lines": []}	{"error": "INVALID_QUANTITY", "cart_lines": []}	PASS	0.053
TC-CRT-005	REQ-02	COND-CRT-02	TS-CRT-02	{"error": "INVALID_QUANTITY", "cart_lines": []}	{"error": "", "cart_lines": [["A", true]]}	FAIL	0.069
TC-CRT-006	REQ-02	COND-CRT-03	TS-CRT-03	{"cart_lines": [["A", 5]], "subtotal": 1000000}	{"cart_lines": [["A", 5]], "subtotal": 1000000, "error": ""}	PASS	0.063
TC-CRT-007	REQ-02	COND-CRT-03	TS-CRT-03	{"cart_lines": [["A", 5], ["B", 1]], "subtotal": 1300000}	{"cart_lines": [["A", 5], ["B", 1]], "subtotal": 1300000, "error": ""}	PASS	0.061
TC-CRT-008	REQ-02	COND-CRT-02	TS-CRT-02	{"error": "INVALID_QUANTITY", "cart_lines": [["A", 2]], "subtota...	{"error": "INVALID_QUANTITY", "cart_lines": [["A", 2]], "subtota...	PASS	0.055
TC-AUT-001	REQ-04	COND-AUT-01	TS-AUT-01	{"status": "PENDING_COD", "orders": 1, "customer_id": "U_...	{"status": "PENDING_COD", "orders": 1, "customer_id": "U_N...	PASS	0.095
TC-AUT-002	REQ-04	COND-AUT-02	TS-AUT-02	{"error": "LOGIN_REQUIRED", "orders": 0, "stock_delta": {}}	{"error": "LOGIN_REQUIRED", "orders": 0, "stock_delta": {}}	PASS	0.06
TC-AUT-003	REQ-04	COND-AUT-02	TS-AUT-02	{"error": "LOGIN_REQUIRED", "orders": 0, "charges": 0, "stoc...	{"error": "LOGIN_REQUIRED", "orders": 0, "charges": 0, "stock_...	PASS	0.097
TC-VCH-001	REQ-07	COND-VCH-01	TS-VCH-01	{"error": "INVALID_VOUCHER", "orders": 0, "stock_delta": {}, ...	{"error": "INVALID_VOUCHER", "orders": 0, "stock_delta": {}, "u...	PASS	0.064
TC-VCH-002	REQ-07	COND-VCH-02	TS-VCH-02	{"error": "VOUCHER_USED", "orders": 0, "stock_delta": {}}	{"error": "VOUCHER_USED", "orders": 0, "stock_delta": {}}	PASS	0.058
TC-VCH-003	REQ-07	COND-VCH-03	TS-VCH-03	{"orders": 1, "discount": 60000, "used_codes": ["SAVE10"]}	{"orders": 1, "discount": 60000, "used_codes": ["SAVE10"], "er...	PASS	0.458
TC-VCH-004	REQ-07	COND-VCH-03	TS-VCH-03	{"error": "PAYMENT_DECLINED", "orders": 0, "used_codes": []}	{"error": "PAYMENT_DECLINED", "orders": 0, "used_codes": []}	PASS	0.142
TC-VCH-005	REQ-07	COND-VCH-03	TS-VCH-03	{"orders": 1, "status": "PAID", "discount": 60000, "used_codes"...	{"orders": 1, "status": "PAID", "discount": 60000, "used_codes":...	PASS	0.183
TC-VCH-006	REQ-07	COND-VCH-02	TS-VCH-02	{"orders": 1, "charges": 1, "used_codes": ["SAVE10"], "attemp...	{"orders": 1, "charges": 1, "used_codes": ["SAVE10"], "attempts"...	PASS	0.17
TC-SHP-001	REQ-09	COND-SHP-01	TS-SHP-01	{"shipping": 30000, "total": 529999}	{"shipping": 30000, "total": 529999, "error": ""}	PASS	0.12
TC-SHP-002	REQ-09	COND-SHP-01	TS-SHP-01	{"shipping": 0, "total": 500000}	{"shipping": 0, "total": 500000, "error": ""}	PASS	0.111
TC-SHP-003	REQ-09	COND-SHP-01	TS-SHP-01	{"shipping": 0, "total": 500001}	{"shipping": 0, "total": 500001, "error": ""}	PASS	0.112
TC-SHP-004	REQ-09	COND-SHP-01	TS-SHP-01	{"shipping": 0, "discount": 60000, "total": 540000}	{"shipping": 0, "discount": 60000, "total": 540000, "error": ""}	PASS	0.123
TC-SHP-005	REQ-09	COND-SHP-02	TS-SHP-02	{"shipping": 50000, "total": 350000}	{"shipping": 50000, "total": 350000, "error": ""}	PASS	0.102
TC-SHP-006	REQ-09	COND-SHP-02	TS-SHP-02	{"shipping": 50000, "total": 1550000}	{"shipping": 50000, "total": 1550000, "error": ""}	PASS	0.102
TC-SHP-007	REQ-09	COND-SHP-03	TS-SHP-03	{"error": "INVALID_AREA", "orders": 0, "stock_delta": {}}	{"error": "INVALID_AREA", "orders": 0, "stock_delta": {}}	PASS	0.09
TC-PAY-001	REQ-10	COND-PAY-01	TS-PAY-01	{"status": "PENDING_COD", "orders": 1, "charges": 0}	{"status": "PENDING_COD", "orders": 1, "charges": 0, "error": ""}	PASS	0.134
TC-PAY-002	REQ-10	COND-PAY-02	TS-PAY-02	{"status": "PAID", "orders": 1, "charges": 1, "charge_amount": ...	{"status": "PAID", "orders": 1, "charges": 1, "charge_amount": 6...	PASS	0.108
TC-PAY-003	REQ-10	COND-PAY-03	TS-PAY-03	{"error": "PAYMENT_DECLINED", "orders": 0, "charges": 0}	{"error": "PAYMENT_DECLINED", "orders": 0, "charges": 0}	PASS	0.1
TC-PAY-004	REQ-10	COND-PAY-02	TS-PAY-02	{"status": "PAID", "charges": 1, "charge_amount": 1400000, "t...	{"status": "PAID", "charges": 1, "charge_amount": 1400000, "to...	PASS	0.119
TC-ATM-001	REQ-15	COND-ATM-01	TS-ATM-01	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}	PASS	0.093
TC-ATM-002	REQ-15	COND-ATM-01	TS-ATM-01	{"error": "OUT_OF_STOCK", "orders": 0, "charges": 0, "stock_de...	{"error": "OUT_OF_STOCK", "orders": 0, "charges": 0, "stock_de...	PASS	0.091
TC-ATM-003	REQ-15	COND-ATM-02	TS-ATM-02	{"orders": 1, "stock_delta": {"A": -20}}	{"orders": 1, "stock_delta": {"A": -20}, "error": ""}	PASS	0.094
TC-ATM-004	REQ-15	COND-ATM-02	TS-ATM-02	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}	PASS	0.084
TC-ATM-005	REQ-15	COND-ATM-03	TS-ATM-01	{"orders": 1, "stock_delta": {"A": -2, "B": -1}}	{"orders": 1, "stock_delta": {"A": -2, "B": -1}, "error": ""}	PASS	0.097
TC-ATM-006	REQ-15	COND-ATM-01	TS-ATM-01	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}	{"error": "OUT_OF_STOCK", "orders": 0, "stock_delta": {}}	PASS	0.083
TC-PMT-001	REQ-16	COND-PMT-01	TS-PMT-01	{"error": "INVALID_PAYMENT_METHOD", "orders": 0}	{"error": "INVALID_PAYMENT_METHOD", "orders": 0}	PASS	0.078
TC-PMT-002	REQ-16	COND-PMT-02	TS-PMT-01	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "charge...	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "charges...	FAIL	0.101
TC-PMT-003	REQ-16	COND-PMT-03	TS-PMT-02	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "stock_...	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "stock_d...	FAIL	0.085
TC-PMT-004	REQ-16	COND-PMT-03	TS-PMT-02	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "stock_...	{"error": "INVALID_PAYMENT_METHOD", "orders": 0, "stock_d...	FAIL	0.083

Hình 13. E08 – Bảng kết quả chi tiết (reports/test_execution.csv)

- Mỗi dòng: TC ↔ REQ/COND/TS, expected_json, actual_result (giá trị quan sát cho đúng các khóa của oracle), status, thời gian. Cùng dữ liệu có trong `test_report.xlsx` sheet Execution và `test_dashboard.html`.

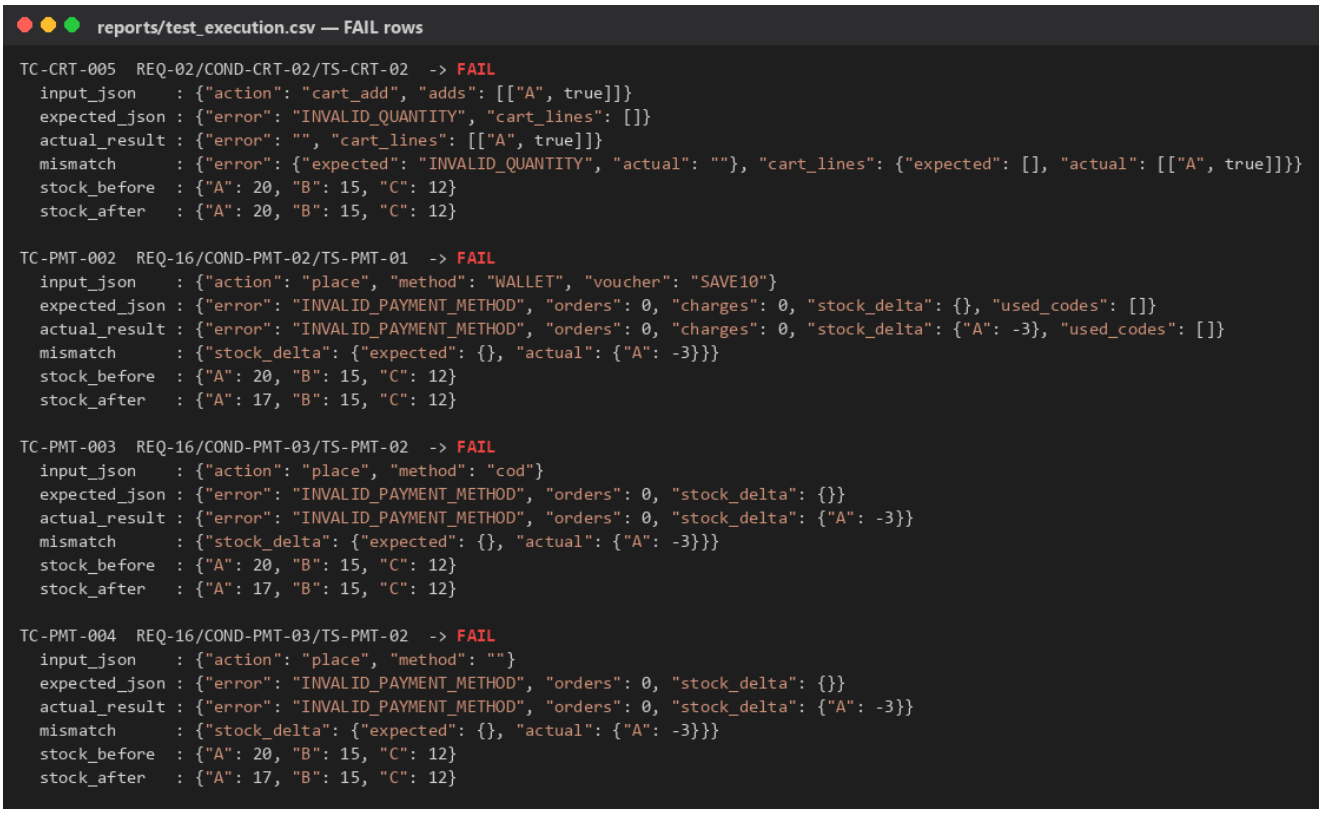
test_report.xlsx — sheet ByRequirement

requirement_id	PASS	FAIL	BLOCKED	TOTAL	PASS_RATE_%
REQ-02	7	1	0	8	87.5
REQ-04	3	0	0	3	100.0
REQ-07	6	0	0	6	100.0
REQ-09	7	0	0	7	100.0
REQ-10	4	0	0	4	100.0
REQ-15	6	0	0	6	100.0
REQ-16	1	3	0	4	25.0

Hình 14. E08 – Sheet ByRequirement của test_report.xlsx

9. Root cause / defect tickets / retest

Quy trình trước khi kết luận bug: (1) đối chiếu expected với acceptance rule ở Mục 3 – expected đúng; (2) chạy lại trên **fixture sạch, ngoài runner** (Hình 19) – lỗi tái hiện 100%; (3) loại trừ lỗi test: fixture nạp dòng giỏ trực tiếp, isolation check 0 khác biệt; (4) đọc code nghiệp vụ để tìm statement vi phạm.



Hình 15. E09 – Các dòng FAIL: input, expected, actual, mismatch, stock trước/sau

DEF-001 – Phương thức thanh toán không hợp lệ bị từ chối nhưng tồn kho vẫn bị trừ

Field	Nội dung
Defect ID / Title	DEF-001 – INVALID_PAYMENT_METHOD nhưng stock bị giữ (không release)
Requirement / TC trace	REQ-16 / COND-PMT-02, COND-PMT-03 / TS-PMT-01, TS-PMT-02 / TC-PMT-002, TC-PMT-003, TC-PMT-004 (TC-PMT-001 PASS vì chỉ kiểm error+orders)
Reproduce	`app,u,cart = fixture("600k")` (A×3) → `app.place(u, cart, method="WALLET", key="R1")` → bắt ValueError → đọc `app.catalog.products["A"].stock`. Lặp lại với `"cod"` và `""`.
Expected vs Actual	Expected: INVALID_PAYMENT_METHOD, orders=0, charges=0, stock A 20 → 20, used_codes=[]. Actual: lỗi đúng, orders=0, charges=0 nhưng **stock A 20 → 17** (stock_delta {A: −3}).
Root cause	`ecommerce_app.py` → `Checkout.place()`: `self.inventory.reserve(cart.lines)` chạy **trước** `self.payments.collect(...)`. Method chỉ được validate bên trong `Payments.collect()` (`if method!="ONLINE": raise ValueError("INVALID_PAYMENT_METHOD")`). Exception thoát khỏi place() mà không có try/except/finally gọi `inventory.release()` → hàng đã giữ không được trả. Vi phạm REQ-16 (request bị từ chối phải không có hiệu lực) và tinh thần nguyên tử của REQ-15/REQ-08.
Severity / Priority	Severity **Major** (lệch tồn kho, có thể khiến sản phẩm "hết hàng" giả; client/API lỗi lặp lại có thể khóa toàn bộ kho). Priority **P1** – sửa trước release vì dễ khai thác và cùng gốc với lỗi rollback khi payment declined (REQ-08).
Suggested fix	(1) Validate method ở đầu `place()` trước mọi side effect; (2) bọc `payments.collect()` trong `try/except` gọi `inventory.release(cart.lines)` rồi `raise` (Hình 20).
Regression	Toàn bộ REQ-16 (TC-PMT-001..004), REQ-10 (TC-PAY-001..004), REQ-15 (TC-ATM-001..006), REQ-07 (TC-VCH-004..006 – có nhánh declined), cùng worked example REQ-08 & REQ-11 của Instruction vì thay đổi thứ tự trong `place()`.

ecommerce_app.py — Checkout.place() (suspect: reserve before collect, no release)

```
76      def place(self,customer,cart,area='inner',voucher=None,method='COD',approved=True,key='REQ-001'):
77          if not customer.authenticated: raise ValueError('LOGIN_REQUIRED')
78          if not cart.lines: raise ValueError('EMPTY_CART')
79          subtotal=cart.subtotal()
80          discount=self.promotions.discount(voucher,subtotal,customer)
81          shipping=self.shipping.calculate(subtotal,area)
82          total=max(D(0),subtotal-discount+shipping)
83          # Reserve stock during the checkout transaction.
84          self.inventory.reserve(cart.lines)
85          if key in self.by_key: return self.orders[self.by_key[key]]
86          # Process the online payment outcome.
87          payment_state=self.payments.collect(total,method,approved,key)
88          self.counter+=1
89          order=Order(f'ORD-{self.counter:04d}',customer.customer_id,total,payment_state,
90                      [(x.sku,x.quantity) for x in cart.lines],discount,shipping,method)
91          self.orders[order.order_id]=order; self.by_key[key]=order.order_id
92          if voucher: customer.used_codes.add(voucher)
93          return order
```

Hình 16. E09 – Checkout.place(): reserve() trước collect(), không có nhánh release

ecommerce_app.py — Payments.collect()

```
63  class Payments:
64      def __init__(self): self.charges=[]
65      def collect(self,amount,method,approved,key):
66          if method=='COD': return 'PENDING_COD'
67          if method!='ONLINE': raise ValueError('INVALID_PAYMENT_METHOD')
68          if not approved: raise ValueError('PAYMENT_DECLINED')
69          self.charges.append({'key':key,'amount':amount})
70          return 'PAID'
```

Hình 17. E09 – Payments.collect(): nơi duy nhất validate method

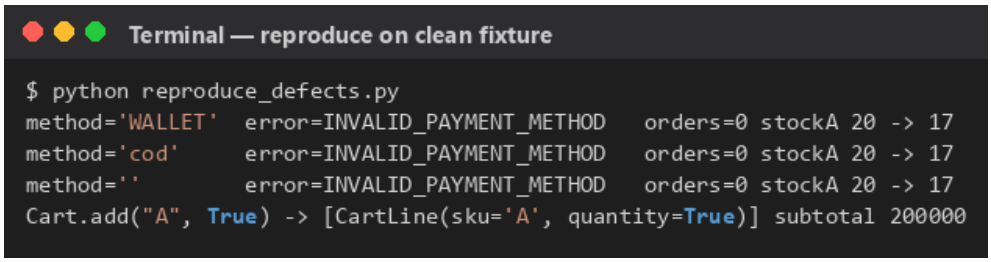
DEF-002 – Cart.add chấp nhận quantity kiểu bool

Field	Nội dung
Defect ID / Title	DEF-002 – `Cart.add("A", True)` được chấp nhận như quantity = 1
Requirement / TC trace	REQ-02 / COND-CRT-02 / TS-CRT-02 / TC-CRT-005
Reproduce	`c = Cart(Catalog()); c.add("A", True)` → `c.lines == [CartLine("A", True)]`, `c.subtotal() == 200000`.
Expected vs Actual	Expected: ValueError INVALID_QUANTITY, cart rỗng. Actual: không lỗi, cart_lines = [["A", true]].
Root cause	`ecommerce_app.py` → `Cart.add()`: `if not isinstance(quantity,int) or quantity<1` – trong Python `bool` là lớp con của `int` nên `isinstance(True, int)` là True và `True < 1` là False → lọt validation. Payload JSON `{"quantity": true}` từ client lỗi sẽ thành 1 sản phẩm.
Severity / Priority	Severity **Minor** (giá trị vẫn là 1, không sai tiền) – Priority **P3**. Cần BA xác nhận (open question REQ-02) trước khi đóng.
Suggested fix & Regression	Thêm `isinstance(quantity, bool)` vào điều kiện từ chối. Regression: TC-CRT-001..008 và các case dùng giỏ hàng (toàn suite, vì Cart dùng ở mọi checkout).

ecommerce_app.py — Cart.add() (isinstance(True, int) is True)

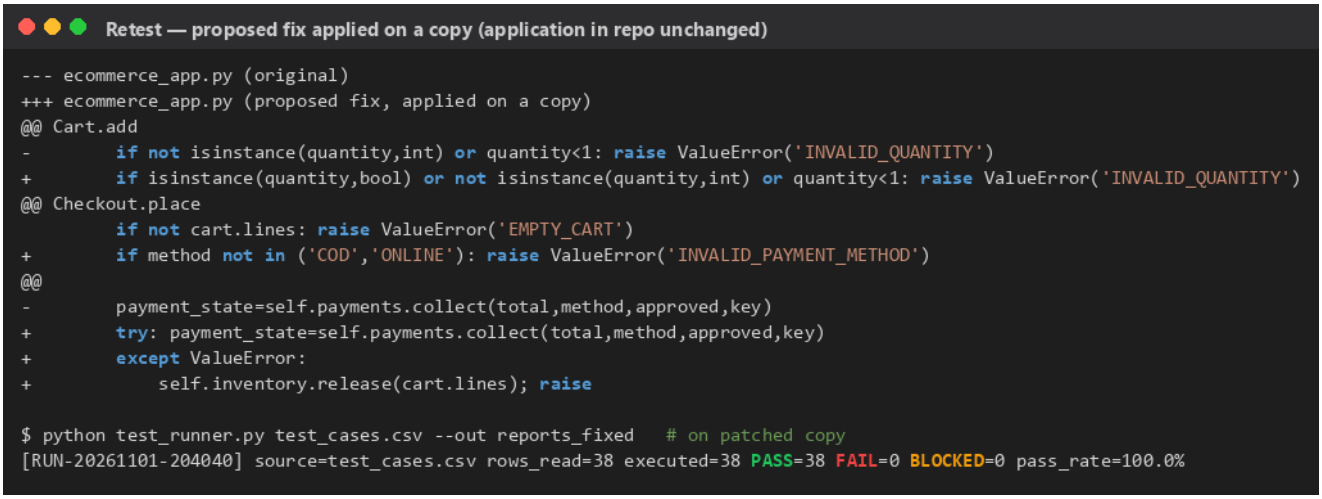
```
31  class Cart:
32      def __init__(self,catalog): self.catalog=catalog; self.lines=[]
33      def add(self,sku,quantity):
34          self.catalog.get(sku)
35          if not isinstance(quantity,int) or quantity<1: raise ValueError('INVALID_QUANTITY')
36          existing=next((x for x in self.lines if x.sku==sku),None)
37          if existing: existing.quantity+=quantity
38          else: self.lines.append(CartLine(sku,quantity))
39      def subtotal(self): return sum((self.catalog.get(x.sku).price*x.quantity for x in self.lines),D(0))
```

Hình 18. E09 – Cart.add(): isinstance(True, int) == True



Hình 19. E09 – Tái hiện trên fixture sạch, ngoài test runner (docs/reproduce_defects.py)

Retest sau khi vá (trên bản sao, không sửa app trong repo)



Hình 20. Bản vá đề xuất và kết quả chạy lại toàn bộ suite: 38/38 PASS

- Bản vá chỉ áp dụng trên bản sao để chứng minh đề xuất đúng; `ecommerce_app.py` trong repo giữ nguyên nên report chính vẫn phản ánh trạng thái thực của ứng dụng.
- Kế hoạch sau khi dev sửa thật: **Sanity** – TC-PMT-002, TC-CRT-005 + 1 case happy path mỗi module; **Regression** – toàn bộ 38 case + 12 case của Instruction; so sánh report trước/sau theo test_case_id.

10. Problem Solving (Phần 3)

3.1. Hai SKU, một SKU thiếu tồn kho

Phân tích nguyên nhân / risk: Giỏ A=2 (đủ), B=100 (stock 15). Theo REQ-15, checkout lỗi thì **stock của A không được thay đổi** – đơn hàng là một giao dịch "tất cả hoặc không". Rủi ro: reserve kiểu "kiểm–trừ" trong cùng một vòng lặp sẽ trừ A trước khi phát hiện B thiếu; rủi ro thứ hai là lỗi xảy ra **sau** reserve (payment) mà không release.

File / function chịu ảnh hưởng: `ecommerce_app.py` → `Inventory.reserve()` (đã đúng: vòng 1 kiểm tra mọi dòng, vòng 2 mới trừ) và `Checkout.place()` (thiếu release khi bước sau reserve thất bại – xem DEF-001).

Thay đổi dự kiến: Giữ thiết kế 2 pha của reserve; thêm try/except quanh các bước sau reserve để release. Test cần kiểm state, không chỉ kiểm exception, và thử cả hai thứ tự dòng.

Test case	Input / bước	Expected	Kết quả
TC-ATM-001	A2_B100, COD	OUT_OF_STOCK; orders 0; stock_delta {}	PASS
TC-ATM-006	B100_A2 (đảo thứ tự)	OUT_OF_STOCK; stock_delta {}	PASS
TC-ATM-002	A2_B100, ONLINE approved	OUT_OF_STOCK; charges 0; stock_delta {}	PASS

Evidence: cột `stock_before/stock_after` trong test_execution.csv ({"A":20,"B":15,"C":12} trước = sau).

3.2. Voucher chỉ đánh dấu used sau thành công

Phân tích nguyên nhân / risk: Khách áp SAVE10, ONLINE bị declined. Nếu `used_codes` thay đổi thì khách **mất voucher dù không có đơn**, lần sau bị VOUCHER_USED – vi phạm REQ-07 ("chỉ đánh dấu used sau khi order thành công"), gây khiếu nại và sai số liệu khuyến mãi.

File / function chịu ảnh hưởng: `Checkout.place()` – dòng `if voucher: customer.used_codes.add(voucher)` nằm sau `payments.collect()` và sau khi tạo Order (đúng); `Promotions.discount()` chỉ kiểm tra, không ghi.

Thay đổi dự kiến: Không cần sửa logic voucher. Đề xuất: trong hệ thống thật, đánh dấu voucher trong cùng transaction với tạo order. Thiết kế sequence test: declined → approved → dùng lại.

Test case	Input / bước	Expected	Kết quả
TC-VCH-004	SAVE10 + ONLINE declined	PAYMENT_DECLINED; used_codes []	**PASS**
TC-VCH-005	declined → approved	#1 DECLINED, #2 PAID, discount 60.000, used [SAVE10]	**PASS**
TC-VCH-006	declined → approved → COD lần 3	#3 VOUCHER_USED; orders 1; charges 1	**PASS**

Evidence: trường `attempts` + `used_codes` trong `observed_json`. Lưu ý: lần declined vẫn làm lệch stock (lỗi REQ-08 đã biết từ Instruction) – nằm ngoài oracle của REQ-07 nên không assert ở đây.

3.3. Mã thanh toán lạ (method="WALLET")

Phân tích nguyên nhân / risk: Thứ tự đúng: (1) xác thực khách → (2) giỏ không rỗng → (3) **validate method** → (4) tính subtotal/discount/shipping → (5) reserve stock → (6) collect payment → (7) tạo order, ghi by_key → (8) đánh dấu voucher. Lỗi mong đợi: `ValueError("INVALID_PAYMENT_METHOD")` ở bước 3, **trước mọi side effect**. Code hiện tại validate ở bước 6 → stock đã bị giữ (DEF-001).

File / function chịu ảnh hưởng: `Checkout.place()`, `Payments.collect()`.

Thay đổi dự kiến: Thêm guard method ở đầu `place()`; release stock nếu collect lỗi. Assertion side effect: `error == INVALID_PAYMENT_METHOD`, `len(orders)==0`, `len(charges)==0`, `stock_before == stock_after`, `used_codes` không đổi, `key not in by_key`.

Test case	Input / bước	Expected	Kết quả
TC-PMT-001	WALLET	INVALID_PAYMENT_METHOD; orders 0	**PASS**
TC-PMT-002	WALLET + SAVE10	+ charges 0; stock_delta {}; used_codes []	**FAIL**
TC-PMT-003/004	"cod" / ""	INVALID_PAYMENT_METHOD; stock_delta {}	**FAIL**

Evidence: Hình 15–17, 19; retest Hình 20.

3.4. Giá sản phẩm thay đổi giữa add-to-cart và checkout

Phân tích nguyên nhân / risk: Requirement hiện tại **chưa nói** chốt giá khi nào. Code: `CartLine` chỉ lưu sku+quantity; `Cart.subtotal()` đọc giá catalog **tại thời điểm checkout** → khách có thể trả giá khác giá đã thấy. Ảnh hưởng dây chuyền: ngưỡng ship 500k (REQ-09), ngưỡng voucher (REQ-06), charge amount (REQ-10).

File / function chịu ảnh hưởng: `Cart.subtotal()`, `CartLine`, `Checkout.place()`.

Thay đổi dự kiến: Câu hỏi BA/PO: (1) Chốt giá lúc add-to-cart hay lúc checkout? (2) Giá tăng có cần khách xác nhận lại? Giá giảm có tự áp dụng? (3) Giá giữ trong bao lâu? (4) Ngưỡng ship/voucher tính theo giá nào? Nếu chốt lúc add: thêm `unit_price` vào CartLine. Điều kiện test: giá tăng / giảm / vượt qua ngưỡng 500k. Expected **chưa xác định được** → case ở trạng thái BLOCKED (chờ BA) thay vì tự đoán.

Test case	Input / bước	Expected	Kết quả
TC-PRC-001 (đề xuất)	A1+B1, đổi giá B 300.000→299.999 trước checkout, inner	Nếu chốt lúc checkout: subtotal 499.999, ship 30.000; nếu chốt lúc add: 500.000, ship 0	**BLOCKED**
TC-PRC-002 (đề xuất)	B×1 300.000 → 350.000, ONLINE	charge_amount = giá đã chốt; cần khách xác nhận nếu tăng	**BLOCKED**

Evidence: câu trả lời BA được ghi vào requirement trước khi chuyển hai case sang CSV.

3.5. Cùng SKU nhập hai lần (A=2 rồi A=3)

Phân tích nguyên nhân / risk: Kỳ vọng: **một** dòng A với quantity 5, subtotal 1.000.000; thứ tự dòng giữ nguyên; lần add lỗi không làm hỏng dòng cũ. Requirement chưa có **giới hạn số lượng** – A có thể cộng dồn vượt tồn kho (20) trong giỏ và chỉ bị chặn khi checkout (OUT_OF_STOCK); cần BA xác nhận có kiểm khi add hay không.

File / function chịu ảnh hưởng: `Cart.add()` (tìm `existing` theo sku rồi cộng dồn), `Inventory.reserve()`.

Thay đổi dự kiến: Nếu BA quy định giới hạn (vd. ≤ stock hoặc ≤ 10/đơn): kiểm tổng quantity sau cộng dồn trong `Cart.add()`; thêm mã lỗi `QUANTITY_LIMIT_EXCEEDED`.

Test case	Input / bước	Expected	Kết quả
TC-CRT-006	add A2, A3	cart_lines [[A,5]], subtotal 1.000.000	PASS
TC-CRT-007	add A2, B1, A3	[[A,5],[B,1]], 1.300.000	PASS
TC-CRT-008	add A2, A0	INVALID_QUANTITY; dòng A2 giữ nguyên	PASS
TC-CRT-009 (đề xuất)	add A15, A6 rồi checkout	Hiện tại: giỏ A21, checkout OUT_OF_STOCK; expected cuối chờ BA	BLOCKED

Evidence: `cart_lines` và `subtotal` trong observed_json.

3.6. Data-driven tests phụ thuộc nhau

Phân tích nguyên nhân / risk: Nguyên nhân: dùng chung object `Checkout/Catalog` giữa các dòng CSV (khởi tạo một lần đầu notebook). Case đổi giá B (fixture 499999), case trừ stock, case ghi `used_codes`, `orders`, `counter`, `by_key` sẽ làm case sau chạy trên state bẩn → kết quả phụ thuộc thứ tự, FAIL "ma", hoặc tệ hơn là PASS giả.

File / function chịu ảnh hưởng: `test_runner.fixture()`, `execute_case()`; phía app: `Catalog.__init__` tạo Product mới mỗi lần (không dùng dict cấp module – điều kiện để isolation hoạt động).

Thay đổi dự kiến: Mỗi dòng gọi `fixture()` riêng tạo Checkout/Customer/Cart mới; không có biến toàn cục giữa các case. Chứng minh: chạy suite theo thứ tự gốc, đảo, xáo trộn (seed cố định) và mỗi case 2 lần → so status theo test_case_id; kiểm `a.catalog is not b.catalog`.

Test case	Input / bước	Expected	Kết quả
TC-ISO-001	Chạy TC-SHP-001 (giá B 299.999) rồi TC-SHP-002	TC-SHP-002 vẫn total 500.000, ship 0	PASS
TC-ISO-002	Suite đảo ngược / xáo trộn / lặp đôi	0 khác biệt status so với lần chạy gốc	PASS

Evidence: Hình 8 (`docs/isolation_check.py`).

3.7. Dashboard báo PASS dù thiếu test

Phân tích nguyên nhân / risk: CSV 20 dòng, 3 dòng parse lỗi bị bỏ qua, 17 PASS → báo "100%" là **sai**: chỉ biết 17/20 = 85% PASS, 3 dòng **chưa được kiểm** (trạng thái không xác định). Dashboard che mất rủi ro, vi phạm tính toàn vẹn của evidence.

File / function chịu ảnh hưởng: `test_runner.read_cases()`, `execute_case()` (khối setup), `run_suite()` (tổng hợp, pass rate).

Thay đổi dự kiến: Lỗi dữ liệu → BLOCKED (không skip); pass rate = PASS / **rows_read**; `assert len(df) == len(cases)`; report hiển thị rows_read, executed, BLOCKED; trùng ID / thiếu cột → dừng suite. Đã hiện thực trong runner.

Test case	Input / bước	Expected	Kết quả
TC-DQ-001	broken_cases.csv: 2 dòng tốt + JSON hỏng + fixture lạ + action lạ	rows_read 5 = executed 5; PASS 2, BLOCKED 3; pass rate 40%	PASS
TC-DQ-002	CSV có 2 dòng trùng test_case_id	Dừng với DUPLICATE_TEST_IDS	PASS
TC-DQ-003	CSV thiếu cột expected_json	Dừng với MISSING_COLUMNS	PASS

Evidence: Hình 7 (self-check). TC-DQ-002/003 là test của runner, chạy bằng cách sửa bản sao CSV.

3.8. Đổi ngưỡng miễn phí vận chuyển 500k → 700k

Phân tích nguyên nhân / risk: Change impact: **REQ-09** đổi nội dung; **COND-SHP-01** biên mới 699.999 / 700.000 / 700.001; **TS-SHP-01** giữ ý nghĩa nhưng dữ liệu đổi; **TC** cần sửa expected: TC-SHP-002 (500.000 → ship 30.000, total 530.000), TC-SHP-003 (500.001 → 30.000), TC-SHP-004 (600k + SAVE10 → ship 30.000, total 570.000); **data set**: thêm fixture 699999/700k/700001 (vd. A2+B1 = 700.000); **code**: `Shipping.calculate` hard-code `D(500000)` → tách hằng số/config `FREE_SHIP_THRESHOLD`. Cảnh báo: ngưỡng voucher SAVE10 (REQ-06) **cũng là 500k nhưng không đổi** – không được gộp chung hằng số.

File / function chịu ảnh hưởng: `Shipping.calculate()`; gián tiếp `Checkout.place()` (total), `Payments` (charge amount).

Thay đổi dự kiến: Regression: toàn bộ REQ-09, REQ-12 (total), REQ-10 (charge amount), REQ-06 (đảm bảo ngưỡng voucher vẫn 500k), TC-PAY-004, TC-SHP-*. Dùng traceability để lọc mọi TC có fixture subtotal 500k–700k.

Test case	Input / bước	Expected	Kết quả
TC-SHP-101 (mới)	inner, 699.999	shipping 30.000	Chờ code mới
TC-SHP-102 (mới)	inner, 700.000	shipping 0	Chờ code mới
TC-SHP-103 (mới)	inner, 700.001	shipping 0	Chờ code mới
TC-SHP-002 (sửa)	inner, 500.000	shipping 30.000 (trước: 0)	Chờ code mới

Evidence: diff CSV (expected cũ/mới) + report trước/sau thay đổi.

11. Interview Questions (Phần 4)

Câu 1. Phân biệt Requirement, Test Condition, Test Scenario, Test Case và Test Execution bằng ví dụ EduCommerce.

Requirement là quy tắc: REQ-09 "nội thành miễn ship nếu subtotal ≥ 500.000". **Test condition** là khía cạnh cần kiểm: COND-SHP-01 "subtotal quanh ngưỡng 500.000". **Test scenario** là tình huống nghiệp vụ (WHAT): TS-SHP-01 "phí ship nội thành tại ngưỡng". **Test case** là dữ liệu + bước + expected cụ thể (HOW): TC-SHP-002 – fixture 500k, inner, COD → shipping 0, total 500.000. **Test execution** là lần chạy thật: runner gọi `place()`, ghi actual, status PASS, thời gian, run ID.

Câu 2. Test oracle là gì? Vì sao không lấy Actual Result của ứng dụng làm Expected?

Oracle là nguồn quyết định kết quả đúng – ở đây là acceptance criteria (và giả định đã được BA xác nhận). Nếu chép actual làm expected thì test luôn PASS và "đóng băng" bug: ví dụ REQ-16, ứng dụng trừ 3 sản phẩm khi method=WALLET; chép actual sẽ biến lỗi DEF-001 thành "đúng". Test khi đó chỉ còn là regression của hành vi hiện tại chứ không kiểm requirement.

Câu 3. Khi nào dùng equivalence partitioning và boundary value analysis trong Pricing/Shipping?

EP khi miền input chia thành các lớp có cùng hành vi: area ∈ {inner, outer, invalid}; subtotal {< 500k, ≥ 500k}; method {COD, ONLINE, khác} – mỗi lớp lấy 1 đại diện (TC-SHP-005 outer 300k). BVA tại điểm chuyển của lớp có thứ tự: 499.999 / 500.000 / 500.001 (TC-SHP-001..003), stock 20/21, quantity 0/1. Lỗi `<=` thay `<` (REQ-06 trong Instruction) chỉ lộ ra ở đúng biên.

Câu 4. Positive vs Negative vs Edge case: ví dụ với Cart.add().

Positive: `add("A", 2)` → dòng A=2 (TC-CRT-006). **Negative**: `add("A", -1)` hoặc `add("Z", 1)` → INVALID_QUANTITY / PRODUCT_UNAVAILABLE (TC-CRT-003). **Edge**: `add("A", 1)` biên dưới hợp lệ, `add("A", 0)` biên dưới không hợp lệ, `add("A", True)` kiểu dữ liệu đặc biệt (TC-CRT-005 – phát hiện DEF-002), add lỗi sau một add thành công (TC-CRT-008).

Câu 5. Fixture là gì? Vì sao cần setup môi trường độc lập cho mỗi test case?

Fixture là đoạn code dựng precondition (object, dữ liệu, state) trước khi chạy test. Trong bài: `fixture(key)` tạo Checkout/Customer/Cart mới với subtotal xác định. Cần độc lập vì ứng dụng có state biến đổi (stock, price, orders, used_codes, by_key); nếu dùng chung, kết quả phụ thuộc thứ tự chạy, không tái hiện được, và một FAIL có thể là do case trước chứ không do sản phẩm.

Câu 6. Test isolation khác reset dữ liệu sau khi test thế nào?

Isolation: mỗi test chạy trên môi trường riêng tạo mới từ đầu → không thể nhiễm state, kể cả khi test trước crash. **Reset/teardown**: dùng chung môi trường và cố gắng hoàn nguyên sau test → nếu test lỗi giữa chừng hoặc reset thiếu một field (vd. quên `by_key` hay giá B) thì state bẩn vẫn lọt sang test sau. Isolation an toàn hơn; reset chỉ dùng khi tạo mới quá tốn kém (DB thật) và phải kèm kiểm tra state đầu vào.

Câu 7. Nếu Python raise ValueError, làm sao phân biệt PASS expected exception và FAIL exception ngoài mong đợi?

Khai báo exception trong oracle: `expected_type=EXCEPTION`, `expected_json.error = "OUT_OF_STOCK"`. Runner bắt `ValueError`, lưu `str(exc)` vào actual rồi so: đúng mã + đúng state (orders 0, stock_delta {}) → PASS; sai mã, không có exception, hoặc exception khác loại (TypeError → `UNEXPECTED:`) → FAIL. Với case RETURN, mọi exception đều là FAIL. Lỗi trong setup/test data → BLOCKED, không phải FAIL.

Câu 8. Tại sao kiểm return value chưa đủ cho Inventory/Payments/Orders?

Vì requirement quy định **side effect**. TC-PMT-001 chỉ kiểm error + orders nên PASS, trong khi TC-PMT-002 kiểm thêm stock thì FAIL – return value giống hệt nhau nhưng kho bị trừ 3. Tương tự: payment declined phải không có charge, retry không được trừ kho lần 2, voucher không bị đánh dấu khi đơn thất bại. Phải chụp state trước/sau (stock, orders, charges, used_codes, by_key).

Câu 9. Giải thích Decimal thay float cho tính tiền và vấn đề rounding.

float là nhị phân nên không biểu diễn chính xác nhiều số thập phân: `0.1 + 0.2 = 0.30000000000000004`; cộng dồn nhiều dòng hoặc nhân % sẽ lệch và so sánh `==` ở ngưỡng (500.000) có thể sai. `Decimal(str(x))` biểu diễn chính xác hệ 10; `money()` dùng `quantize(..., ROUND_HALF_UP)` để làm tròn đúng quy tắc kế toán (vd. 10% của 499.995 = 49.999,5 → 50.000). Phải tạo Decimal từ chuỗi, không từ float.

Câu 10. Data-driven testing là gì? Một dòng CSV liên kết với function kiểm thử thế nào?

Tách **dữ liệu test** (CSV) khỏi **logic test** (runner): thêm case = thêm dòng. Liên kết: `fixture_key` → hàm `fixture()` dựng state; `input_json.action` → dispatcher chọn `act_cart_add`/`act_place`/`act_place_sequence` → gọi `Cart.add`/`Checkout.place` với tham số trong input_json; `expected_type`/`expected_json` → bộ so sánh; `test_case_id`/`requirement_id` → traceability trong report. Action/fixture chưa được hiện thực → BLOCKED, không tự đoán.

Câu 11. Traceability Matrix phục vụ impact analysis và coverage ra sao?

Coverage: nhìn ma trận REQ→COND→TS→TC→RUN biết requirement/condition nào chưa có case hoặc case chưa chạy/BLOCKED (bài: 7/7 REQ, 20/20 COND). **Impact analysis**: khi REQ-09 đổi ngưỡng 700k (3.8) lọc ngay mọi COND/TS/TC liên quan để sửa expected và lập regression; khi một TC FAIL truy ngược REQ để đánh giá ảnh hưởng nghiệp vụ.

Câu 12. Priority của Test Case và Severity của Defect khác nhau thế nào?

Test case priority = thứ tự nên chạy/thiết kế dựa trên rủi ro và tần suất (P0 thanh toán, tồn kho; P3 kiểu dữ liệu hiếm). **Defect severity** = mức độ ảnh hưởng kỹ thuật/nghiệp vụ của lỗi (DEF-001 Major – lệch kho; DEF-002 Minor). Defect còn có **priority sửa** do PO quyết định (deadline, khách hàng). Case P0 có thể tìm ra lỗi Minor và ngược lại.

Câu 13. Unit vs Integration vs System: testcase dựa trên Checkout.place() thuộc mức nào?

`Cart.add()`, `Shipping.calculate()` gọi riêng là **unit**. `Checkout.place()` phối hợp thật Promotions, Shipping, Inventory, Payments, Catalog (không mock) nên là **integration/service-level** (component integration). Chưa phải **system test** vì không có UI/API, DB, cổng thanh toán, mạng thật – Instruction cũng ghi rõ đây là lab unit/service-level.

Câu 14. Giải thích state transition COD/ONLINE và xác nhận PAID.

Từ "đang checkout": **COD** → order `PENDING_COD` (chưa thu tiền, không có charge; sẽ chuyển PAID khi giao hàng thu tiền – ngoài phạm vi lab). **ONLINE + approved** → `PAID` ngay, ghi 1 charge = total. **ONLINE + declined** → không có order (trạng thái cuối "rejected"), không charge, stock phải được trả lại. Method khác → bị từ chối trước khi vào máy trạng thái. Xác nhận PAID: assert `order.status=="PAID"` **và** `len(charges)==1`, `charge.amount==order.total` (TC-PAY-002, TC-PAY-004).

Câu 15. Idempotency có nghĩa gì trong checkout? Vì sao retry cần kiểm cả side effects?

Cùng một request (cùng idempotency key) gửi nhiều lần cho kết quả như gửi một lần: trả lại order cũ, không tạo order mới, không trừ kho, không charge thêm. Retry thường xảy ra khi client timeout – nếu chỉ kiểm "không có order mới" sẽ bỏ sót lỗi trừ kho lần 2 (đúng lỗi REQ-11 trong Instruction: `reserve()` chạy trước khi tra `by_key`). Vì vậy phải kiểm len(orders), stock, charges sau mỗi lần retry.

Câu 16. FAIL do sản phẩm vs FAIL do test script/fixture: xác định thế nào?

(1) Kiểm lại expected so với requirement; (2) tái hiện bằng đoạn code tối thiểu ngoài runner trên fixture sạch (Hình 19); (3) chạy case đơn lẻ và theo thứ tự khác (isolation check); (4) kiểm precondition thực tế (stock_before, giá) có đúng fixture; (5) đọc code để chỉ ra statement vi phạm. Chỉ khi cả 5 bước trỏ về code sản phẩm mới mở defect; lỗi dữ liệu/setup được runner tách thành BLOCKED/HARNESS_ERROR.

Câu 17. Gradio dashboard có thay thế evidence/raw execution log không? Vì sao?

Không. Dashboard là **view tổng hợp** – có thể lọc, làm tròn, và link `share=True` chỉ tạm thời, phụ thuộc runtime Colab. Evidence cần dữ liệu thô bền vững, truy vết được: test_execution.csv (expected/actual/state/timestamp/run_id), output console, report Excel, mã nguồn ở commit cụ thể. Dashboard dùng để giao tiếp, raw log dùng để kiểm chứng và audit.

Câu 18. Sau khi sửa bug, nên chạy Sanity và Regression thế nào?

Sanity/confirmation: chạy lại đúng các case FAIL (TC-PMT-002..004, TC-CRT-005) + vài happy path chính để xác nhận bản build dùng được. **Regression**: dựa traceability chọn mọi case liên quan vùng code bị sửa (`Checkout.place` → REQ-04/07/09/10/15/16 và REQ-08/11 của Instruction), lý tưởng là toàn bộ suite vì chạy tự động < 1s; so sánh report trước/sau theo test_case_id, không được có PASS → FAIL mới.

Câu 19. Một repository có những gì để người khác tái hiện kết quả Colab?

Mã nguồn SUT (`ecommerce_app.py`) và runner, dữ liệu test (`test_cases.csv`), `requirements.txt` có phiên bản, notebook Colab, README với các lệnh chạy từ môi trường sạch, report mẫu kèm run ID để đối chiếu, script tái hiện defect, tài liệu thiết kế (REQ/COND/TS/TC). Không commit credentials/token; không phụ thuộc file chỉ có trong `/content` của Colab.

Câu 20. Nếu chỉ có thời gian kiểm thử 5 case, em chọn theo risk như thế nào?

Risk = khả năng lỗi × mức ảnh hưởng tiền/kho/uy tín. Chọn: (1) TC-PAY-002 ONLINE approved → PAID & charge = total (thu tiền đúng); (2) TC-PAY-003 declined → không order, không charge; (3) TC-ATM-001 một SKU thiếu → không trừ kho phần còn lại; (4) TC-SHP-002 biên 500.000 (mọi đơn bị ảnh hưởng); (5) TC-PMT-002 method lạ không side effect. Ưu tiên case kiểm **state** để một case phủ nhiều rủi ro.

11.1. Code Interview

A – Exception oracle

Vấn đề: runner coi **mọi** exception là FAIL, nên test thiết kế để mong đợi `ValueError("INVALID_QUANTITY")` luôn FAIL – đây là lỗi **thiết kế test/runner**, không phải bug sản phẩm. Sửa: (1) test design khai báo exception là một phần của expected (`expected_type=EXCEPTION`, mã lỗi cụ thể); (2) runner bắt đúng loại `ValueError`, so mã lỗi, và vẫn kiểm state; exception khác loại hoặc không có exception mới là FAIL.

```
# Sai
try:
    cart.add(sku, qty); status = 'PASS'
except Exception:
```

```
    status = 'FAIL'

# Đúng
error = ''
try:
    cart.add(sku, qty)
except ValueError as exc:            # chỉ lỗi nghiệp vụ; TypeError... vẫn nổi lên/FAIL
    error = str(exc)
if row['expected_type'] == 'EXCEPTION':
    passed = error == expected['error'] and cart_lines(cart) == expected['cart_lines']
else:
    passed = error == '' and cart_lines(cart) == expected['cart_lines']
```

B – False positive

Chỉ kiểm `order.order_id is not None` là **assertion yếu / thiếu oracle** dẫn tới **false positive** (test PASS trong khi sản phẩm sai): đơn tạo được nhưng có thể trừ kho sai, total sai, charge sai. Bổ sung:

```
before = {k: p.stock for k, p in app.catalog.products.items()}
order  = app.place(user, cart, area='inner', voucher='SAVE10', method='ONLINE', approved=True,
key='K1')
after  = {k: p.stock for k, p in app.catalog.products.items()}

assert order.order_id.startswith('ORD-') and len(app.orders) == 1
assert after['A'] == before['A'] - 3 and after['B'] == before['B']      # stock trừ đúng 1 lần, đúng
SKU
assert order.discount == D(60000) and order.shipping == D(0)
assert order.total == D(600000) - order.discount + order.shipping == D(540000)
assert order.total >= 0
assert order.status == 'PAID' and len(app.payments.charges) == 1
assert app.payments.charges[0]['amount'] == order.total                 # thu đúng số tiền
assert order.customer_id == 'U_NEW' and order.lines == [('A', 3)]
assert 'SAVE10' in user.used_codes and app.by_key['K1'] == order.order_id
```

C – Shared fixture

Rủi ro: fixture tạo một lần đầu notebook rồi dùng cho mọi case → stock bị trừ dồn, giá B 299.999 lan sang case sau, `used_codes` khiến case voucher sau báo VOUCHER_USED, `by_key`/`counter` làm order ID và idempotency sai; kết quả phụ thuộc thứ tự cell và không tái hiện được. Cách kiểm: chạy suite theo thứ tự khác/xáo trộn/lặp đôi và so status; in `id(app)` hoặc stock trước mỗi case; chạy một case đơn lẻ so với khi chạy trong suite. Cách sửa: tạo fixture **trong** `execute_case()` cho từng dòng (factory, không phải biến toàn cục), không dùng mutable default/biến cấp module trong app, và assert precondition đầu case.

```
# Sai – dùng chung
app, user, cart = fixture('600k')
for row in rows: run(row, app, user, cart)

# Đúng – mỗi dòng một môi trường mới
for row in rows:
    app, user, cart = fixture(row['fixture_key'])
    assert {k: p.stock for k, p in app.catalog.products.items()} == {'A': 20, 'B': 15, 'C': 12}
    run(row, app, user, cart)
```

12. Self-review

Hạng mục	Trạng thái
≥5 REQ ngoài REQ-06/08/11	Hoàn thành – 7 REQ (REQ-02, 04, 07, 09, 10, 15, 16)
≥10 condition, ≥10 scenario, ≥15 TC, ≥5 negative/boundary	Hoàn thành – 20 COND, 18 TS, 38 TC, 20 negative + nhiều boundary
Traceability REQ→COND→TS→TC→RUN	Hoàn thành – Mục 4.3 + sheet Traceability
Thực thi tự động, PASS/FAIL/BLOCKED, actual,	Hoàn thành – 38 case, 34/4/0

Hạng mục	Trạng thái
evidence, thời gian	
Code test: fixture, reader, dispatcher/assert, exception, export	Hoàn thành – Mục 6
Root cause cho FAIL	Hoàn thành – DEF-001, DEF-002 + retest bản vá
Report CSV/Excel + dashboard HTML/Gradio	Hoàn thành – reports/, dashboard_gradio.py
Phần 3 & Phần 4	Hoàn thành – 8 tình huống, 20 câu + 3 code review
Làm thêm	Isolation check, data-completeness self-check, retest bản vá, tái hiện defect ngoài runner, test_design.xlsx
Còn thiếu / cần sinh viên tự làm	Chụp ảnh chạy trên Colab (E07) và link Gradio; push repo lên GitHub và điền URL; điền thông tin sinh viên

Remaining risks (chưa được kiểm thử trong phạm vi bài):

- Stock không được trả lại khi ONLINE declined (REQ-08 – đã biết từ Instruction, quan sát thấy ở bước #1 của TC-VCH-005 nhưng không assert vì ngoài REQ đã chọn).
- Chốt giá khi giá sản phẩm thay đổi (3.4) và giới hạn số lượng trong giỏ (3.5) chưa có oracle – chờ BA.
- Concurrency (hai checkout đồng thời tranh cùng tồn kho), lỗi mạng/timeout của cổng thanh toán, persistence – ngoài phạm vi lab in-memory.
- Sản phẩm inactive (REQ-01), giỏ rỗng (REQ-14) và cách làm tròn discount (REQ-03/06) không thuộc REQ đã chọn.

GitHub: https://github.com/binhbuithanhk5-a11y/STLab02_EduCommerce